BCA 6th Semester Project Report

UniEvent (UniPass): University Event Management System

Submitted in partial fulfillment of the requirements for the degree of Bachelor of Computer Applications

Student Name 1: _____

Roll Number 1: _____

Student Name 2 (if applicable): _____

Roll Number 2: _____

Internal Guide: _____

Academic Session: 2025-2026

Department / College Name: _____

Certificate

This is to certify that the project report entitled **UniEvent (UniPass): University Event Management System** has been prepared by the above named student(s) under the guidance of the undersigned in partial fulfillment of the BCA 6th Semester requirements. The work embodied in this report is based on the implemented software project available in the submitted source repository and has been examined for academic presentation.

Signature of Internal Guide: _____

Signature of Head of Department: _____

Date: _____ Place: _____

Declaration

We hereby declare that the project report titled **UniEvent (UniPass): University Event Management System** is our original work carried out for the BCA 6th Semester project submission. The report has been prepared from the implemented project source code available in the workspace and has not been submitted elsewhere for any other academic award. References used in preparing this report have been acknowledged in the bibliography.

Student Signature 1: _____

Student Signature 2: _____

Acknowledgement

The development of UniEvent was made possible through continuous learning in full-stack web development, database design, and software engineering practices. We express sincere gratitude to our internal project guide, department faculty, and all peers who helped by reviewing requirements, testing workflows, and providing feedback on usability. We also acknowledge the institutional setting that motivated the practical problem solved by this project: the need for a secure and efficient system for managing university events, registrations, attendance, and certificates.

Abstract

UniEvent (UniPass) is a dynamic university event management platform implemented with Next.js, TypeScript, Node.js, Express, and MongoDB. The system allows organizers to create and manage events, publish them to students, verify participant entry using QR-coded tickets, and generate formal participation certificates. Students can browse events, register online, obtain a downloadable digital ticket, and later access earned certificates. The system addresses the limitations of manual event handling by introducing role-based access, seat-limit control, real-time verification, attendance persistence, and server-generated PDFs. The project demonstrates a practical full-stack solution suitable for academic institutions and aligns with the BCA final semester requirement of producing an interactive database-driven application.

Index

Chapter / Section	Page No.
Introduction to the Project / Problem Assigned	1
Objective(s)	3
Requirement Analysis	4
Problem Analysis	5
Requirement Specification Document	6
Software Design	8
System Design	9
Architectural Design (DFD's)	10
User Interface Design	11
Component Level / Detailed Level Design (Functional Design)	15
Coding	17
Programming Approach Used: Top Down / Bottom Up	18

Chapter / Section	Page No.
Source Code	19
Screenshots of UI	43
Testing	47
Test Cases & Test Criteria	48
Implementation & Evaluation of Project	49
Containing Maintenance Measures	50
Conclusion	51
Future Scope of the Project	52
Annexure	53
Supplementary Technical Note 1	58
Supplementary Technical Note 2	59
Bibliography	60

Introduction to the Project / Problem Assigned

UniEvent, also referred to as UniPass in the code repository, is a dynamic and interactive university event management system designed for academic institutions where multiple departments regularly organize workshops, orientations, fresher programmes, farewell events, expert talks, and technical activities. The project solves a real administrative problem: event information is often scattered across notice boards, spreadsheets, informal messaging groups, and manual paper-based attendance sheets. This causes poor visibility for students, duplicated registrations, slow entry verification, and weak reporting for organizers.

The implemented system offers two clearly separated user roles. A student can discover published events, filter them by type or department, register online, obtain a digital ticket containing a QR code, view ticket status, and later download a certificate for eligible events. An organizer can create and manage events, control publication status, upload a banner, verify participants at entry using either a camera scanner or manual ticket ID entry, monitor registrations, and trigger automatic certificate generation when an event is completed.

The problem assigned for this project can therefore be stated as the need to design a secure, paper-light, responsive, role-based platform that digitizes the full lifecycle of a university event. The lifecycle begins with event definition, continues through publication and registration, and concludes with entry validation, attendance preservation, and certificate issue. The application is dynamic because all information is stored in a database and retrieved on demand through APIs; it is interactive because users actively search, register, verify, and download artifacts through a responsive web interface.

The system is particularly relevant for institutions that run both open campus events and department-specific activities. In the current implementation, public events can be attended by a broad student audience, whereas departmental events enforce eligibility checks by comparing a student's department with the event's assigned department. This makes the software practically useful for universities where access rules differ by event category and where organizers require operational control without sacrificing user convenience.

Functional Highlights

- Dynamic event creation, update, publication, completion, and deletion by organizer accounts.
- Secure student registration with automatic ticket generation and seat-limit enforcement.

- QR-based ticket verification with protection against duplicate or invalid entries.
- Attendance recording at the moment of successful ticket validation.
- Automatic PDF ticket generation and downloadable event certificates.
- Role-based authentication using JSON Web Tokens and password hashing.

Objective(s)

To develop a web-based university event management application that is fully dynamic, responsive, and role-aware.

To provide students with a single portal for discovering, filtering, and registering for university events.

To generate unique digital tickets containing QR codes so that every registration results in a traceable entry token.

To enable organizers to manage event data, registration limits, status transitions, and participant verification from one dashboard.

To automate attendance and certificate workflows so that administrative effort is reduced after the event day.

To maintain the security, integrity, and confidentiality of user and event records through authentication, validation, and input sanitization.

To demonstrate full-stack development capability using Next.js, TypeScript, Node.js, Express, MongoDB, PDF generation, and QR technology.

Requirement Analysis

Requirement analysis was performed by studying the operational needs of two major stakeholders: students and organizers. Students need timely information, fast registration, certainty that their seat is reserved, and a reliable method to show proof of registration at the event venue. Organizers need a controlled workflow for publishing events, preventing overbooking, validating entries, and preserving post-event participation records.

The software also has institutional requirements. It must support both public and departmental events, preserve accountability by associating events with the responsible organizer, and avoid personal information leakage inside QR codes. The repository implementation satisfies this requirement by encoding only the ticket identifier inside the QR artifact. The detailed participant information is looked up server-side only after a scan is submitted for verification.

Non-functional requirements were equally important. The project needed a modern interface, compatibility with ordinary laptops and browsers used by students, secure API access, straightforward deployment, maintainable source code structure, and room for extension. The existing application structure reflects this: the frontend and backend are separated, the backend follows a controller-model-route-service pattern, and the frontend organizes pages, components, hooks, and context providers in a modular form.

Stakeholder Requirements

Stakeholder	Primary Needs
Student	Discover events, filter upcoming events, register once, store ticket, preview/download PDFs, a
Organizer	Create events, upload banners, monitor registrations, verify tickets, complete events, generate
Institution	Accurate attendance records, role separation, reduced manual paperwork, better traceability a
System Administrator	Simple deployment, maintainable codebase, manageable environment variables, dependable

Problem Analysis

In a typical manual process, event information is published through multiple channels without a single source of truth. Registration may occur on paper or through generic forms, leading to duplicate entries, invalid student submissions, and no integrated seat counter. When the event starts, volunteers often check names against printed lists, which is slow and error-prone. Attendance records created in this way are hard to archive and even harder to connect back to certificate generation.

A second problem is the absence of end-to-end traceability. If a student claims registration but the organizer cannot quickly verify it, both user confidence and event throughput suffer. Manual checking at the venue creates queues. The same issue appears after the event when organizers need to know who actually attended instead of who merely registered. Without a digital event system, participation certificates may be delayed or distributed to ineligible participants.

The final problem area concerns security and integrity. Shared spreadsheets expose sensitive student data, and weak authentication can allow unauthorized edits to event details. UniEvent addresses these challenges through user authentication, role-based access checks, validation middleware, rate limiting on authentication routes, and controlled business rules for event registration and ticket use. The system therefore solves both operational inefficiency and data governance shortcomings.

Requirement Specification Document

Introduction to SRS

The Software Requirement Specification for UniEvent defines what the system must do, what constraints it must obey, and how the software will support the target academic environment. It acts as a communication bridge between the conceptual problem and the final implementation. In this report, the SRS is aligned with the repository's current implementation rather than a hypothetical version, so every major requirement described here can be traced to an existing page, endpoint, model, or service.

The SRS is divided into general description and specific requirements. The general description explains the role of the system, its user classes, major functions, assumptions, and operational boundaries. The specific requirements then list the actual input, output, functional, and environmental needs. This structure ensures the report remains consistent with academic documentation standards while still being practical for development and maintenance.

General Description of Project

UniEvent is a web application accessed through a browser. The system presents a landing page, authentication pages, dashboards for two roles, profile screens, ticket and certificate preview modals, and organizer controls for event lifecycle management. The backend exposes RESTful endpoints for authentication, event management, ticket operations, attendance, certificates, file handling, and user profile maintenance.

The major operating characteristics are as follows. Students interact mainly with browse, register, preview, and download functions. Organizers interact mainly with create, update, publish, verify, and complete-event functions. The application depends on MongoDB to persist operational records and uses server-side PDF generation for formal ticket and certificate outputs. QR generation is integrated into the registration pipeline so that no extra manual step is required after booking.

Important assumptions include continuous browser availability, a working database connection, correct environment variable configuration, and users possessing valid credentials. Important constraints include role rules, one-registration-per-student-per-event enforced by a compound index on tickets, and certificate availability only when the event is configured to generate certificates and marked completed.

Specific Requirements

Category	Requirement Description
Inputs	Signup details, login credentials, event details, seat limit, date, time, venue, optional event banne
Outputs	Published event lists, registration confirmation, QR-coded ticket PDF, entry verification result, att
Functional	Authentication, event CRUD, filtering, registration, ticket status handling, certificate generation, fi
Validation	Email format checks, password length rules, department restriction for departmental events, dup
Security	JWT authentication, bcrypt password hashing, sanitized textual inputs, limited auth route reques

Hardware and Software Requirements

Requirement Type	Specification
Client Hardware	Standard laptop/desktop with minimum 4 GB RAM, modern browser, internet or LAN connect
Server Hardware	Development machine with Node.js runtime and sufficient storage for database plus generate
Frontend Software	Next.js 14, React 18, TypeScript, Tailwind CSS, React Query, Axios.
Backend Software	Node.js, Express.js, MongoDB, Mongoose, JWT, bcryptjs, QRCode, PDFKit, Sharp.
Development Tools	VS Code or equivalent editor, npm, Git, local MongoDB or MongoDB Atlas.
Operating Environment	Windows, Linux, or macOS systems capable of running Node.js 18+.

Software Design

The software design follows a separation-of-concerns principle. The frontend is responsible for presentation, navigation, local interaction states, and API invocation. The backend manages authentication, validation, core business logic, persistence, and generation of formal assets such as ticket PDFs and certificates. This separation improves maintainability because changes in presentation logic do not directly alter persistence logic and vice versa.

Within the backend, the route-controller-model-service approach has been followed. Routes define URI structure and validation middleware; controllers coordinate request-specific operations; models define data structure and indexing; services encapsulate reusable logic such as certificate generation, email handling, file storage, and image processing; utility modules handle QR and PDF generation. On the frontend, reusable UI components like buttons, modals, skeletons, and preview dialogs reduce duplication and encourage consistent behavior across pages.

The design is intentionally scalable for an academic project. Even though the current deployment may be a single Node.js service and one database, the module boundaries already support later enhancements such as notifications, admin approval workflows, analytics exports, or a mobile client. The codebase therefore demonstrates both immediate functionality and future-oriented design discipline.

System Design

At the system level, UniEvent is a three-part solution composed of a client interface, an application server, and persistent storage. Users access the application through the Next.js frontend. Frontend pages use authenticated API calls to the Express backend. The backend validates user identity, checks business rules, writes or retrieves MongoDB documents, and returns structured JSON responses. Ticket and certificate artifacts are produced on the server so that document generation remains trusted and consistent.

The system is event-centric. Every operational flow is anchored to an event record and connected entities such as tickets, attendance entries, and certificates. The ticket entity acts as the bridge between registration and attendance. Its status field enables state transition from unused to used or cancelled. The attendance entity records actual participation. The certificate entity records issued proof after completion. This sequence gives the system a coherent lifecycle and makes reporting straightforward.

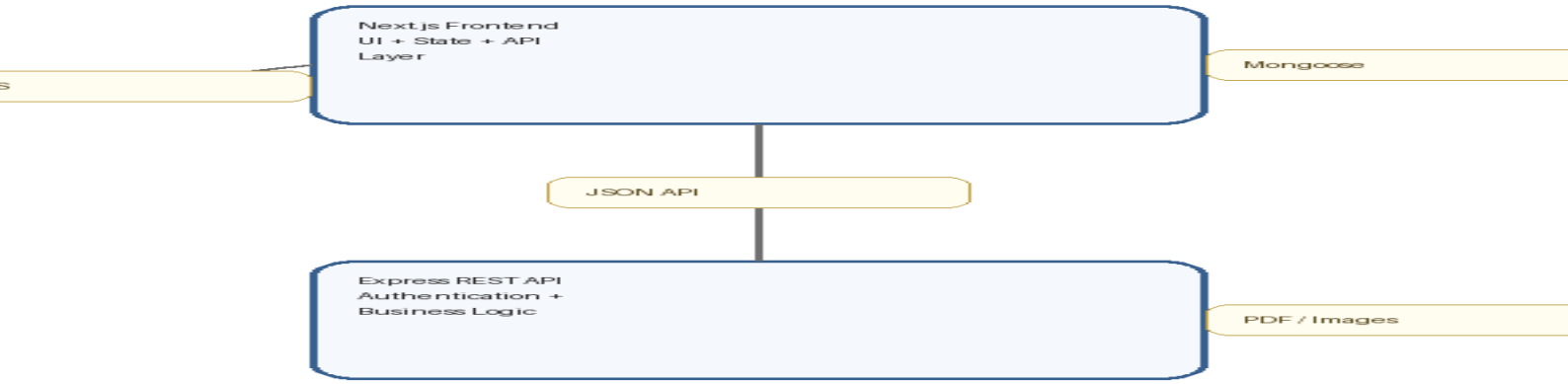


Figure 1: High-level architecture showing the browser client, Next.js frontend, Express API, MongoDB store, and file-generation layer.

Architectural Design (DFD's)

The context-level DFD treats the entire application as one process exchanging information with students, organizers, and persistent storage. Students send registration and viewing requests; organizers send event creation and verification requests; the system writes and reads all durable information through the database. At the next level, the system can be decomposed into authentication management, event management, registration and ticket processing, verification and attendance management, and certificate generation.

Authentication management receives signup, login, forgot-password, and profile requests. Event management handles event creation, listing, update, deletion, and registration statistics. Registration and ticket processing creates unique ticket records, generates QR content, and exposes ticket download or preview operations. Verification and attendance management validates scan payloads and persists attendance. Certificate generation checks event completion state and creates downloadable certificate artifacts for qualified attendees.

Figure 2: Context-level DFD illustrating the major actors and the central UniEvent system.

User Interface Design

The user interface uses a restrained neutral design language with clear typography, rounded cards, soft borders, and strong action buttons. The landing page establishes the product purpose immediately and directs a user into student or organizer onboarding. Dashboard layouts prioritize discoverability and low-friction interactions. For example, the student dashboard provides top-level counters, tabbed content areas, filters, and one-click download actions; the organizer dashboard places event lifecycle actions close to each event card so that publication, scan, completion, and editing tasks remain fast.

The UI is responsive. Cards flow into fewer columns on smaller displays, modal widths adapt to available space, and interactive controls remain visible without unnecessary nesting. This matters because the project guidelines explicitly emphasize practical project execution and live demonstration. A responsive interface makes the software easier to demonstrate in viva-voce settings, whether using a laptop, projector, or smaller personal device.

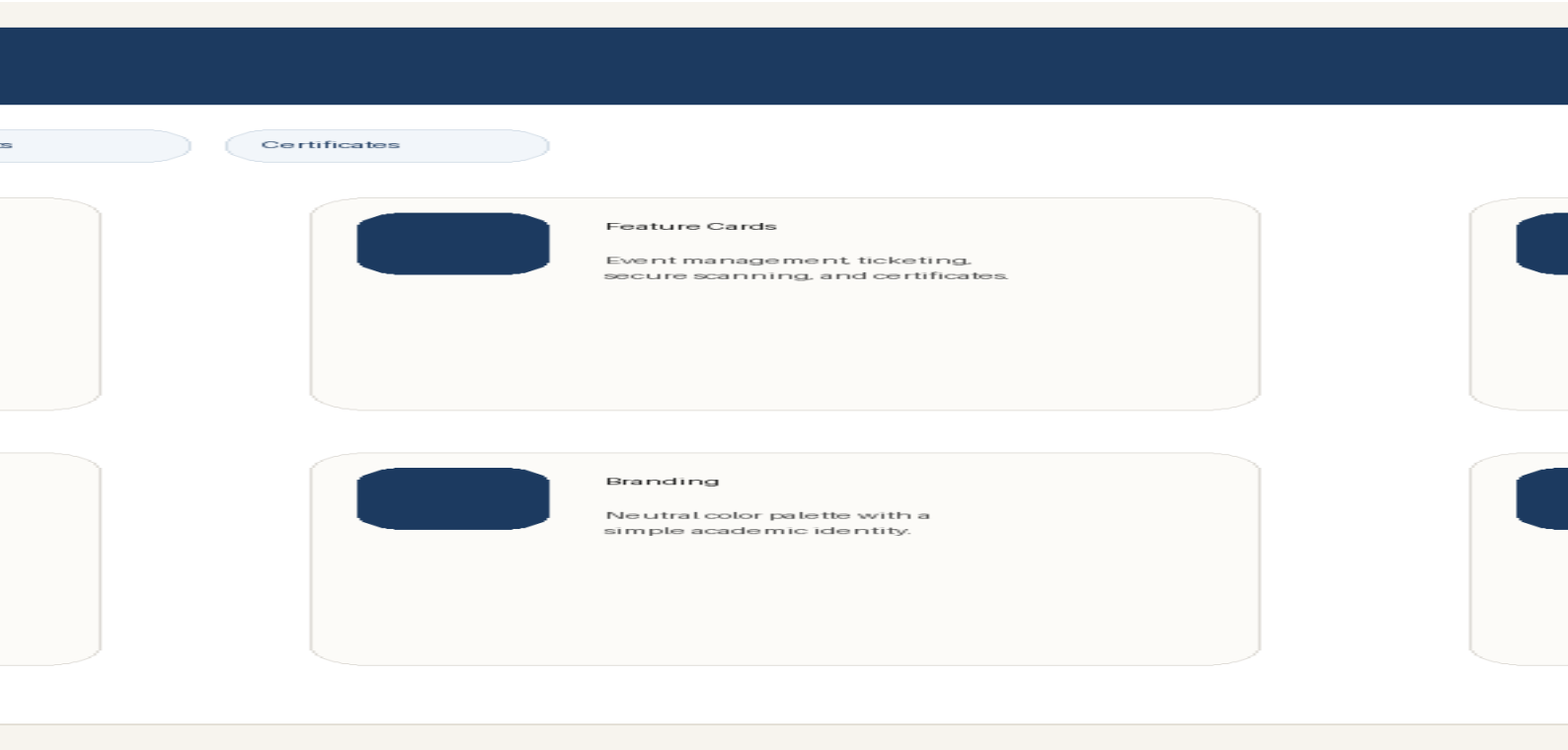


Figure 3: Representative view of the landing page implemented in `frontend/src/app/page.tsx`.

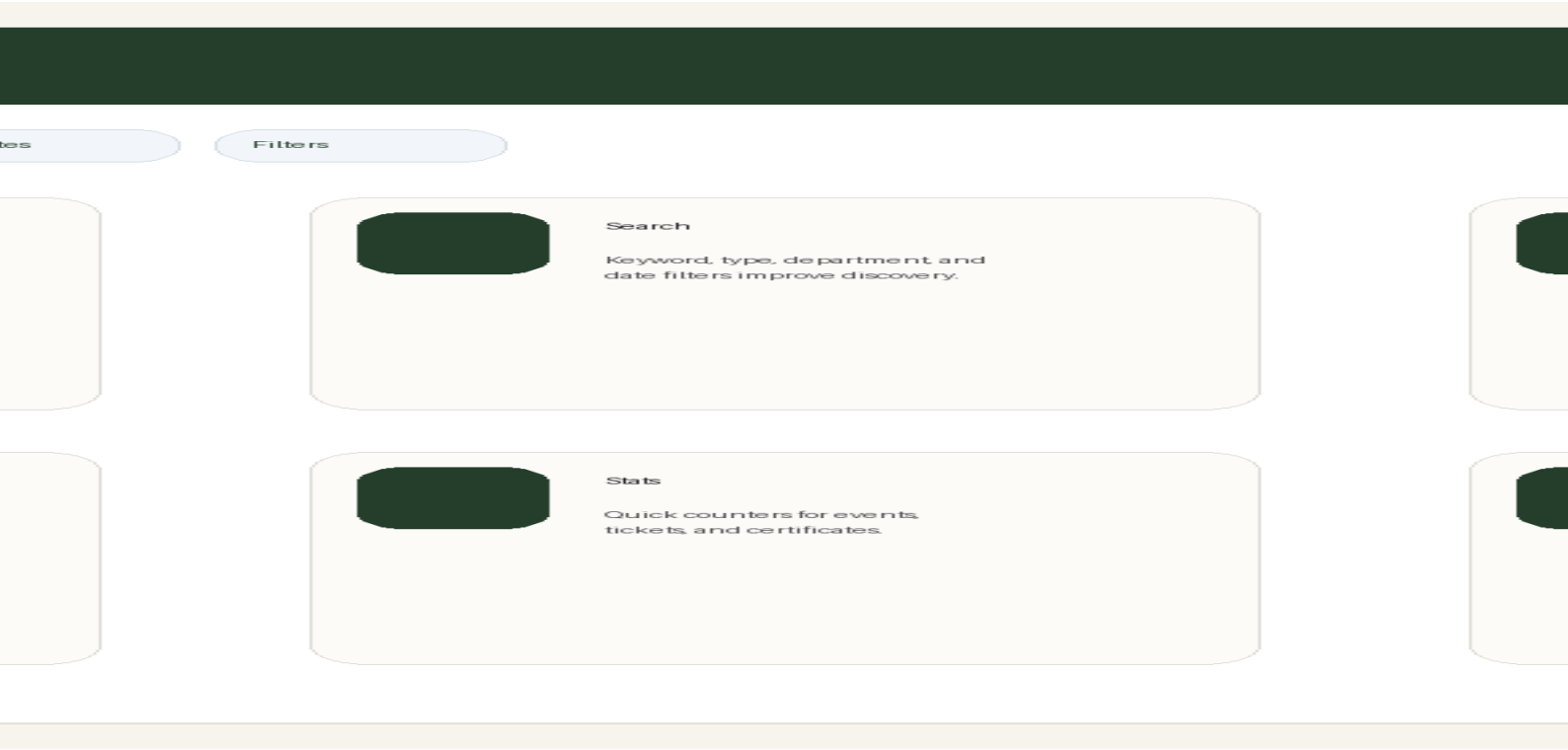


Figure 4: Representative student dashboard showing event discovery, tickets, and certificates.

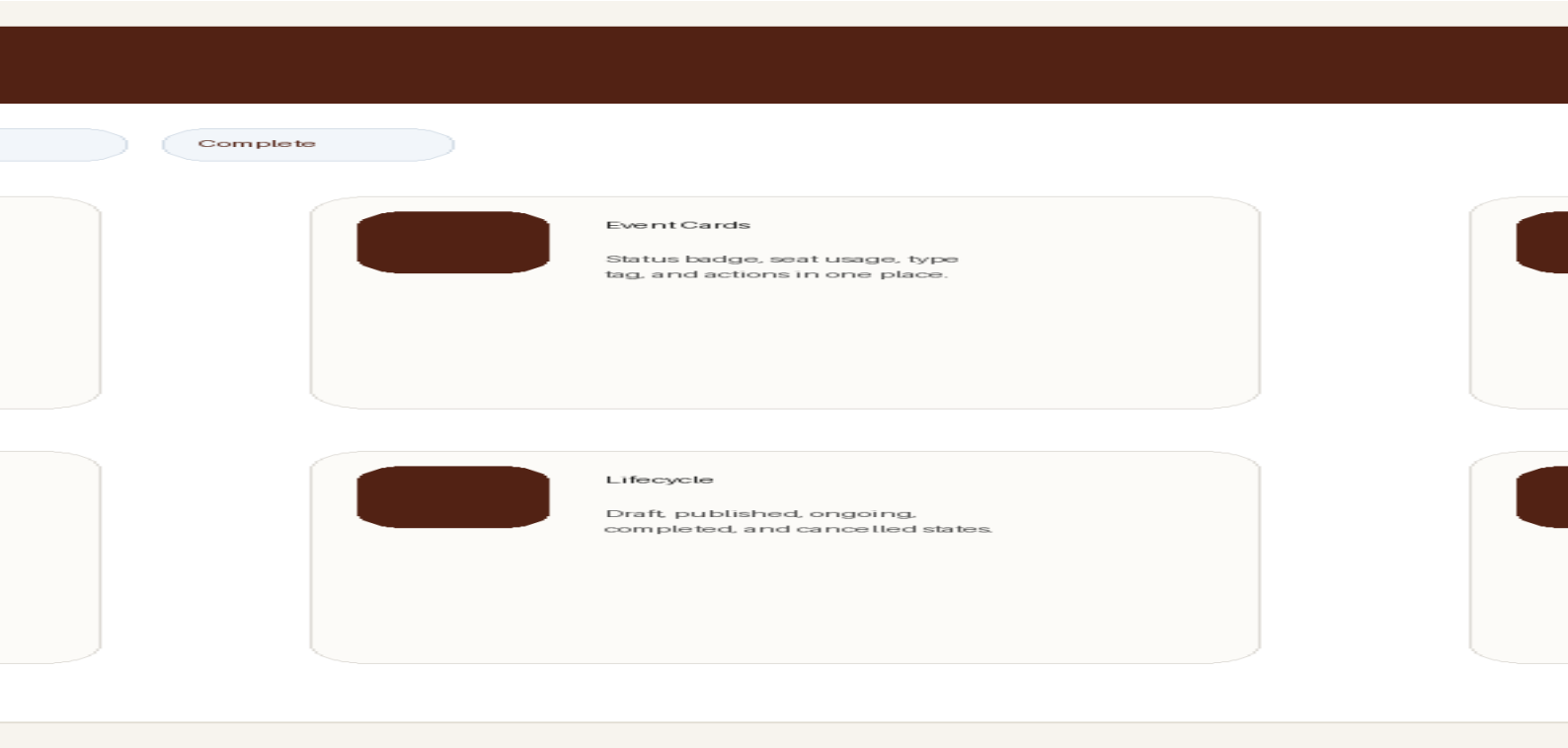


Figure 5: Representative organizer dashboard showing event administration and verification controls.

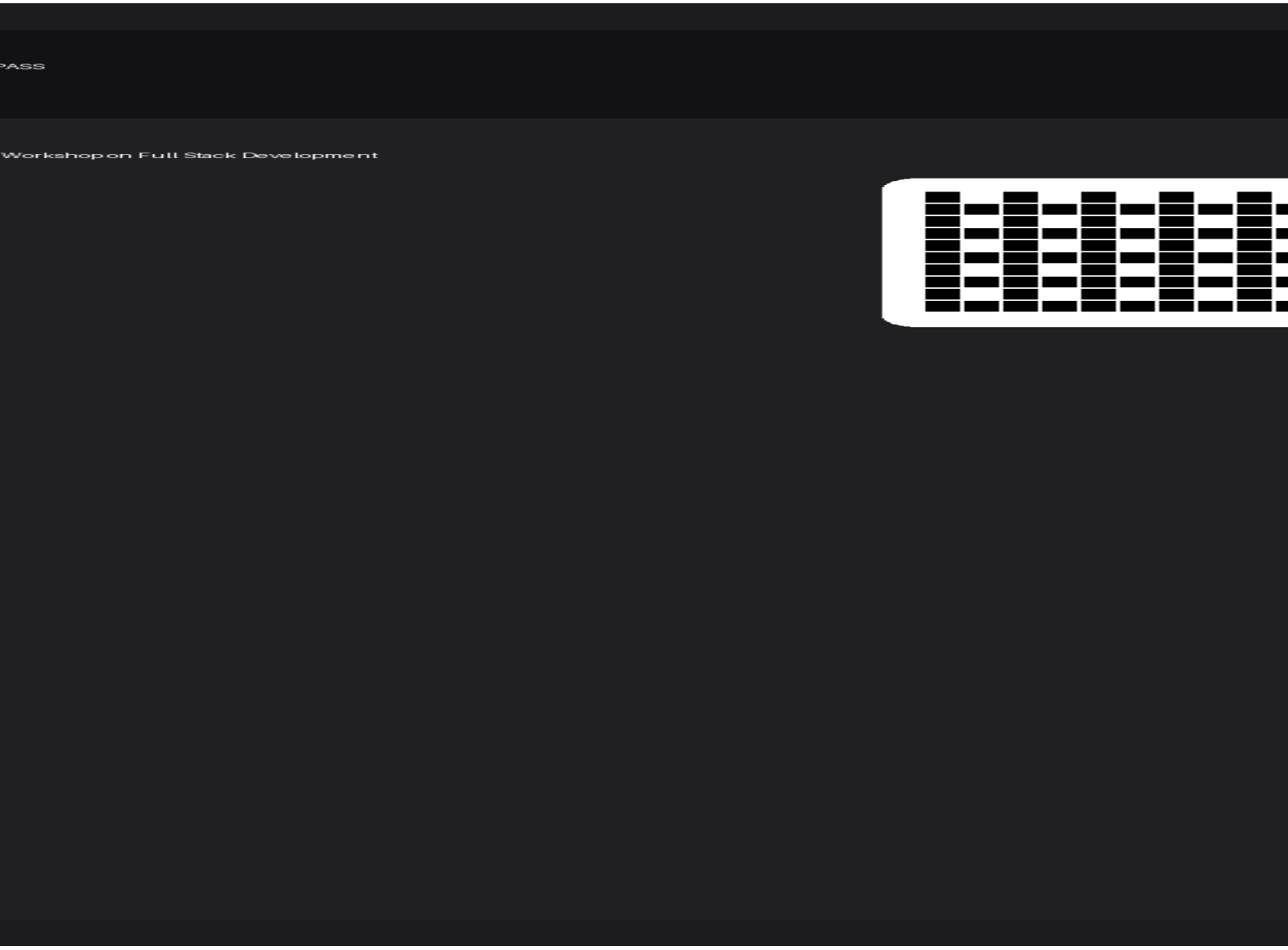


Figure 6: Ticket preview inspired by the implemented movie-style PDF ticket generator.

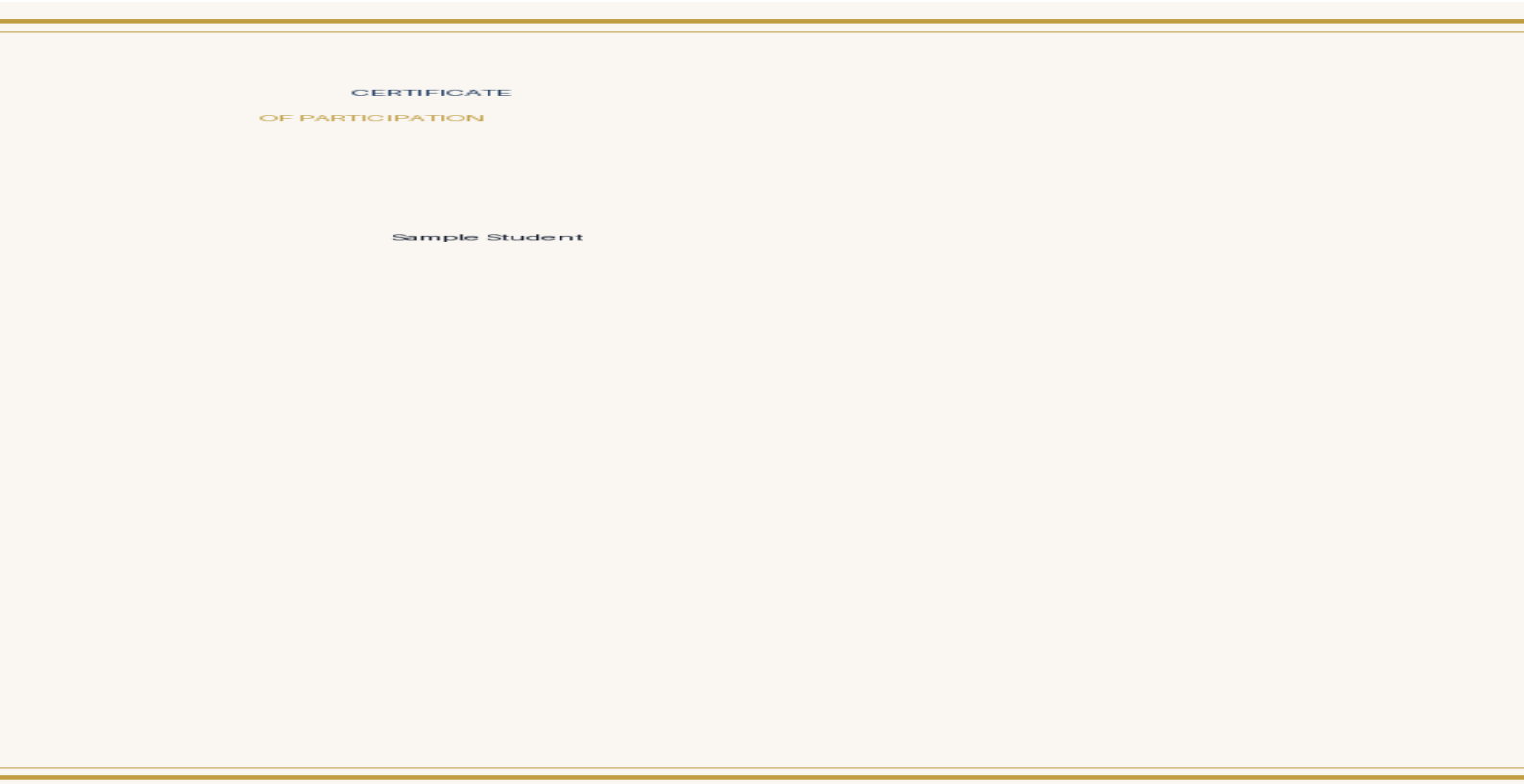


Figure 7: Certificate preview inspired by the implemented landscape PDF certificate generator.

Component Level / Detailed Level Design (Functional Design)

The authentication component validates signup and login payloads, normalizes email addresses, sanitizes user-entered text, hashes passwords before persistence, issues JWT tokens after login, and exposes profile retrieval. The design keeps sensitive values such as passwords and reset tokens out of normal query results.

The event component handles creation, listing, update, and deletion. It also supports filters like event type, department, search, date range, and pagination. Organizer-specific behavior is embedded into the event-list API so that organizers see their own events while students and public users see only published items. The component also controls automatic certificate generation when an event transitions to completed and certificate mode is enabled.

The ticket component handles registration, ticket retrieval, PDF generation, QR verification, and cancellation. It checks registration eligibility, future date constraints, departmental restrictions, duplicate registrations, and seat availability before issuing a ticket. Verification is a multi-step workflow that confirms ticket existence, event match, event ownership, and ticket status before changing the ticket to used and recording attendance.

The certificate component aggregates attendance information after event completion and issues a unique certificate record per student-event pair. The certificate PDF uses an A4 landscape format and formal visual design, making the output suitable for academic documentation or workshop participation evidence.

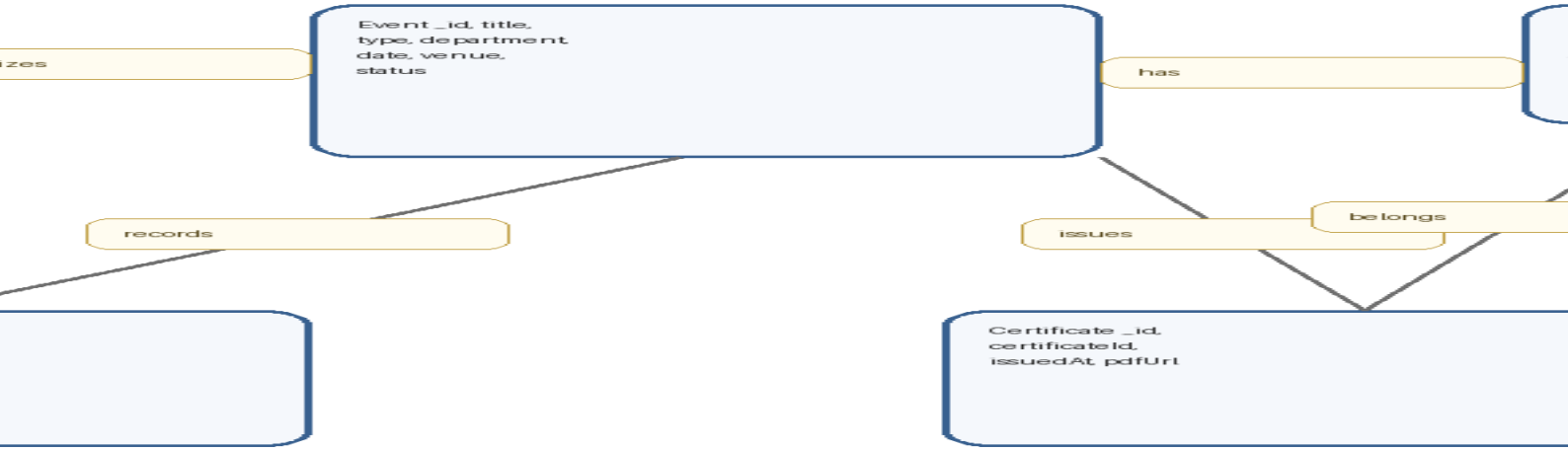


Figure 8: Entity relationship view of the primary data model used by the application.

Coding

The coding phase followed the design defined above and used modular, commented source files. The frontend uses TypeScript for typed props, typed API payload handling, and improved maintainability. The backend uses JavaScript with clear module boundaries and explanatory comments in controllers, models, and utilities. Naming conventions are descriptive enough to make the project suitable for academic review and maintenance after submission.

Coding standards used in the project include clear route separation, defensive validation, explicit status enums, readable JSX card composition, context-driven auth state, and reusable helper methods for formatting, downloading files, and constructing image URLs. Security-sensitive code paths such as authentication, ticket verification, and password reset are designed with validation and authorization checks before business actions are executed.

Programming Approach Used: Top Down / Bottom Up

The development approach used in UniEvent is best described as a hybrid of top-down and bottom-up programming. Top-down design is visible in the way the overall solution was decomposed into roles, modules, API areas, and dashboard flows before the lower-level code was implemented. The major user journeys such as signup, login, event creation, registration, verification, and certificate issue were defined first and then broken into component and endpoint responsibilities.

Bottom-up programming is visible in the creation of reusable low-level building blocks such as models, UI components, utility methods, and middleware. For example, the project defines separate button, modal, skeleton, and card components on the frontend; similarly, the backend separates QR generation, PDF generation, file storage, and validation middleware into reusable modules. This hybrid strategy is appropriate because it keeps the user-facing features coherent while also building maintainable foundations for repeated use.

Source Code

Representative Backend Source Code

The following excerpts demonstrate the structure and style of the implemented backend. The extracts are taken directly from the working repository and show how models, controllers, and utilities support the functional requirements described earlier.

User model with password hashing

```
/**
 * User Model
 * Stores student and organizer information
 */

const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');
const { roles, password: passwordConfig } = require('../config/auth.config');

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Name is required'],
    trim: true,
    maxlength: [100, 'Name cannot exceed 100 characters']
  },
  email: {
    type: String,
    required: [true, 'Email is required'],
    unique: true,
    lowercase: true,
    trim: true,
    match: [/^\S+@\S+\.\S+$/, 'Please provide a valid email']
  },
  password: {
    type: String,
    required: [true, 'Password is required'],
    minlength: [passwordConfig.minLength, `Password must be at least ${passwordConfig.minLength} characters`]
    select: false // Don't include password in queries by default
  },
  rollNumber: {
    type: String,
    trim: true,
    // Required for students, optional for organizers
    sparse: true
  },
  department: {
    type: String,
    required: [true, 'Department is required'],
    trim: true
  },
  role: {
    type: String,
    enum: Object.values(roles),
    default: roles.STUDENT
  },
  phone: {
    type: String,
    trim: true,
    default: null
  },
  profilePhotoUrl: {
    type: String,
    default: null
  },
  profilePhotoFileId: {
    type: mongoose.Schema.Types.ObjectId,
    default: null
  },
  idCardUrl: {
    type: String,
```

```
    default: null
  },
  isActive: {
    type: Boolean,
    default: true
  },

  // Password reset (hashed token only)
  resetPasswordToken: {
    type: String,
    default: null,
    select: false
  },
  resetPasswordExpires: {
    type: Date,
    default: null,
    select: false
  }
}, {
  timestamps: true
});

// Index for efficient queries
// userSchema.index({ email: 1 });
// userSchema.index({ rollNumber: 1 });
userSchema.index({ department: 1, role: 1 });

/**
 * Hash password before saving
 */
userSchema.pre('save', async function(next) {
  // Only hash if password is modified
  if (!this.isModified('password')) {
    return next();
  }

  try {
    const salt = await bcrypt.genSalt(passwordConfig.saltRounds);
    this.password = await bcrypt.hash(this.password, salt);
    next();
  } catch (error) {
    next(error);
  }
});

/**
 * Compare password for login
 * @param {string} candidatePassword - Password to compare
 * @returns {Promise<boolean>} - True if password matches
 */
userSchema.methods.comparePassword = async function(candidatePassword) {
  return bcrypt.compare(candidatePassword, this.password);
};
```

Event model with status and seat logic

```
/**
 * Event Model
 * Stores university event information
 */

const mongoose = require('mongoose');

// Event types
const EVENT_TYPES = {
  PUBLIC: 'public',        // Freshers, Farewell - open to all
  DEPARTMENTAL: 'departmental'  // Workshops - department specific
};

// Event status
const EVENT_STATUS = {
  DRAFT: 'draft',
  PUBLISHED: 'published',
  ONGOING: 'ongoing',
  COMPLETED: 'completed',
  CANCELLED: 'cancelled'
```

```
};

const eventSchema = new mongoose.Schema({
  title: {
    type: String,
    required: [true, 'Event title is required'],
    trim: true,
    maxlength: [200, 'Title cannot exceed 200 characters']
  },
  description: {
    type: String,
    required: [true, 'Event description is required'],
    maxlength: [2000, 'Description cannot exceed 2000 characters']
  },
  organizerId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: [true, 'Organizer is required']
  },
  eventType: {
    type: String,
    enum: Object.values(EVENT_TYPES),
    required: [true, 'Event type is required']
  },
  department: {
    type: String,
    required: [true, 'Department is required'],
    trim: true
  },
  seatLimit: {
    type: Number,
    required: [true, 'Seat limit is required'],
    min: [1, 'Seat limit must be at least 1']
  },
  registeredCount: {
    type: Number,
    default: 0,
    min: 0
  },
  date: {
    type: Date,
    required: [true, 'Event date is required']
  },
  time: {
    type: String,
    required: [true, 'Event time is required']
  },
  venue: {
    type: String,
    required: [true, 'Venue is required'],
    trim: true
  },
  status: {
    type: String,
    enum: Object.values(EVENT_STATUS),
    default: EVENT_STATUS.DRAFT
  },
  bannerUrl: {
    type: String,
    default: null
  },
  bannerFileId: {
    type: mongoose.Schema.Types.ObjectId,
    default: null
  },
  enableCertificates: {
    type: Boolean,
    default: false  // Enable certificate generation for workshops
  }
}, {
  timestamps: true
});

// Indexes for efficient queries
eventSchema.index({ date: 1, status: 1 });
eventSchema.index({ organizerId: 1 });
eventSchema.index({ department: 1, eventType: 1 });

```

```
// Search indexes (used for keyword search)
// Note: even if we use regex for search, a text index is useful for future optimizations.
eventSchema.index({ title: 'text', description: 'text', venue: 'text' });

// Virtual for checking if seats are available
eventSchema.virtual('seatsAvailable').get(function() {
  return this.seatLimit - this.registeredCount;
```

Ticket controller registration and verification logic

```
/**
 * Ticket Controller
 * Handles ticket generation, QR verification, and entry management
 */

const Event = require('../models/Event.model');
const Ticket = require('../models/Ticket.model');
const Attendance = require('../models/Attendance.model');
const { generateQRCode } = require('../utils/qrGenerator');
const { generateTicketPDF } = require('../utils/pdfGenerator');

/**
 * @route   POST /api/tickets/register/:eventId
 * @desc    Register for an event and generate ticket
 * @access  Private (Student)
 */
const registerForEvent = async (req, res, next) => {
  try {
    const { eventId } = req.params;

    // Find event
    const event = await Event.findById(eventId);
    if (!event) {
      return res.status(404).json({
        success: false,
        message: 'Event not found'
      });
    }

    // Check if event is published
    if (event.status !== Event.EVENT_STATUS.PUBLISHED) {
      return res.status(400).json({
        success: false,
        message: 'Registration is not open for this event'
      });
    }

    // Check if seats are available
    if (event.registeredCount >= event.seatLimit) {
      return res.status(400).json({
        success: false,
        message: 'No seats available'
      });
    }

    // Check if event is in the future
    if (new Date(event.date) < new Date()) {
      return res.status(400).json({
        success: false,
        message: 'Registration closed for past events'
      });
    }

    // Check for departmental event eligibility
    if (event.eventType === 'departmental' &&
        event.department !== req.user.department) {
      return res.status(403).json({
        success: false,
        message: 'This event is only for students of ' + event.department
      });
    }

    // Check if already registered
    const existingTicket = await Ticket.findOne({
      eventId,
      userId: req.user._id
```

```
    });

    if (existingTicket) {
      return res.status(400).json({
        success: false,
        message: 'You are already registered for this event'
      });
    }

    // Create ticket with unique ID
    const ticketData = {
      eventId,
      userId: req.user._id,
      qrCode: '' // Will be set after ticket creation
    };

    const ticket = new Ticket(ticketData);

    // Generate QR code containing ONLY the ticket ID (security best practice)
    const qrCode = await generateQRCode(ticket.ticketId);
    ticket.qrCode = qrCode;

    await ticket.save();

    // Increment registered count
    await Event.findByIdAndUpdate(eventId, {
      $inc: { registeredCount: 1 }
    });

    // Populate for response
    await ticket.populate('eventId', 'title date time venue bannerUrl department eventType');
    await ticket.populate('userId', 'name rollNumber department');

    res.status(201).json({
      success: true,
      message: 'Registration successful',
      data: { ticket }
    });
  } catch (error) {
    next(error);
  }
};

/**
 * @route   GET /api/tickets/my-tickets
 * @desc    Get all tickets for current user
 * @access  Private
 */
const getMyTickets = async (req, res, next) => {
  try {
    const { page, limit } = req.query;

    const hasPagination = page !== undefined || limit !== undefined;
    const parsedPage = Math.max(1, parseInt(page || '1', 10));
    const parsedLimit = Math.min(100, Math.max(1, parseInt(limit || '10', 10)));

    const baseQuery = { userId: req.user._id };

    const total = await Ticket.countDocuments(baseQuery);

    let cursor = Ticket.find(baseQuery)
      .populate('eventId', 'title date time venue status bannerUrl department eventType')
      .sort({ createdAt: -1 });

    if (hasPagination) {
      cursor = cursor.skip((parsedPage - 1) * parsedLimit).limit(parsedLimit);
    }

    const tickets = await cursor;

    const totalPages = hasPagination ? Math.max(1, Math.ceil(total / parsedLimit)) : 1;

    res.json({
      success: true,
      count: tickets.length,
      total,
      page: hasPagination ? parsedPage : 1,
      totalPages,
```

```
145         limit: hasPagination ? parsedLimit : total,
146         data: { tickets }
147       });
148     } catch (error) {
149       next(error);
150     }
151   };
152
153   /**
154    * @route   GET /api/tickets/:ticketId
155    * @desc    Get ticket by ID
156    * @access  Private
157    */
158   const getTicketById = async (req, res, next) => {
159     try {
160       const ticket = await Ticket.findOne({ ticketId: req.params.ticketId })
161         .populate('eventId')
162         .populate('userId', 'name rollNumber department');
163
164       if (!ticket) {
165         return res.status(404).json({
166           success: false,
167           message: 'Ticket not found'
168         });
169       }
170
171       // Only ticket owner can view full ticket details
172       if (ticket.userId._id.toString() !== req.user._id.toString() &&
173           req.user.role !== 'organizer') {
174         return res.status(403).json({
175           success: false,
176           message: 'Not authorized to view this ticket'
177         });
178       }
179
180       res.json({
181         success: true,
182         data: { ticket }
183       });
184     } catch (error) {
185       next(error);
186     }
187   };
188
189   /**
190    * @route   POST /api/tickets/verify
191    * @desc    Verify ticket QR code at event entry
192    * @access  Private (Organizer only)
193    *
194    * This is the main entry verification endpoint:
195    * 1. Scans QR code (contains only ticketId)
196    * 2. Validates ticket exists and is unused
197    * 3. Validates ticket is for correct event
198    * 4. Marks ticket as USED
199    * 5. Records attendance
200    */
201   const verifyTicket = async (req, res, next) => {
202     try {
203       const { ticketId, eventId } = req.body;
204
205       if (!ticketId || !eventId) {
206         return res.status(400).json({
207           success: false,
208           message: 'Ticket ID and Event ID are required'
209         });
210       }
211
212       // Find ticket
213       const ticket = await Ticket.findOne({ ticketId })
214         .populate('userId', 'name rollNumber department')
215         .populate('eventId', 'title organizerId');
216
217       if (!ticket) {
218         return res.status(404).json({
219           success: false,
220           message: 'Invalid ticket',
```

Event controller lifecycle logic

```
/**
 * Event Controller
 * Handles event CRUD operations
 */

const Event = require('../models/Event.model');
const Ticket = require('../models/Ticket.model');
const { roles } = require('../config/auth.config');
const certificateService = require('../services/certificateService');
const imageService = require('../services/imageService');

/**
 * @route   POST /api/events
 * @desc    Create a new event
 * @access  Private (Organizer only)
 */
const createEvent = async (req, res, next) => {
  try {
    const {
      title,
      description,
      eventType,
      department,
      seatLimit,
      date,
      time,
      venue,
      enableCertificates
    } = req.body;

    // Create event first
    const event = await Event.create({
      title,
      description,
      organizerId: req.user._id,
      eventType,
      department,
      seatLimit,
      date,
      time,
      venue,
      enableCertificates: enableCertificates === 'true' || enableCertificates === true
    });

    // Handle banner image if uploaded
    if (req.file) {
      try {
        const result = await imageService.processEventBanner(
          req.file.buffer,
          event._id.toString(),
          req.file.mimetype
        );
        event.bannerFileId = result.fileId;
        event.bannerUrl = result.url;
        await event.save();
      } catch (imgError) {
        console.error('Error processing banner image:', imgError);
        // Event is created, just without banner
      }
    }

    res.status(201).json({
      success: true,
      message: 'Event created successfully',
      data: { event }
    });
  } catch (error) {
    next(error);
  }
};

/**
 * @route   GET /api/events
 * @desc    Get all published events (for students) or organizer's events
 * @access  Public/Private
 */
```

```
const getEvents = async (req, res, next) => {
  try {
    const {
      status,
      department,
      eventType,
      type,
      upcoming,
      search,
      dateFrom,
      dateTo,
      page,
      limit,
    } = req.query;

    const parsedPage = Math.max(1, parseInt(page || '1', 10));
    const hasPagination = page !== undefined || limit !== undefined;
    const parsedLimit = Math.min(100, Math.max(1, parseInt(limit || '10', 10)));

    let query = {};

    // If user is logged in and is an organizer, show their events
    if (req.user && req.user.role === roles.ORGANIZER) {
      query.organizerId = req.user._id;
      if (status) query.status = status;
    } else {
      // For students/public, show only published events
      query.status = Event.EVENT_STATUS.PUBLISHED;
    }

    // Filter by department
    if (department) {
      query.department = department;
    }

    // Filter by event type (support both `eventType` and `type` for compatibility)
    const effectiveType = eventType || type;
    if (effectiveType) {
      query.eventType = effectiveType;
    }

    // Date filters
    const dateQuery = {};
    if (upcoming === 'true') {
      dateQuery.$gte = new Date();
    }
    if (dateFrom) {
      const d = new Date(dateFrom);
      if (!isNaN(d.getTime())) dateQuery.$gte = d;
    }
    if (dateTo) {
      const d = new Date(dateTo);
      if (!isNaN(d.getTime())) dateQuery.$lte = d;
    }
    if (Object.keys(dateQuery).length > 0) {
      query.date = dateQuery;
    }

    // Keyword search (case-insensitive regex)
    if (search) {
      const escaped = String(search).replace(/[.*+?^${}()|[\]\\]/g, '\\$&');
      const regex = new RegExp(escaped, 'i');
      query.$or = [
        { title: regex },
        { description: regex },
        { venue: regex },
      ];
    }

    const total = await Event.countDocuments(query);

    let cursor = Event.find(query)
      .populate('organizerId', 'name email department')
      .sort({ date: 1 });

    if (hasPagination) {
      cursor = cursor.skip((parsedPage - 1) * parsedLimit).limit(parsedLimit);
    }
```

```

    const events = await cursor;

    const totalPages = hasPagination ? Math.max(1, Math.ceil(total / parsedLimit)) : 1;

    // Backward compatible: keep data.events
    res.json({
      success: true,
      count: events.length,
      total,
      page: hasPagination ? parsedPage : 1,
      totalPages,
      limit: hasPagination ? parsedLimit : total,
      data: { events }
    });
  } catch (error) {
    next(error);
  }
};

/**
 * @route   GET /api/events/:id
 * @desc    Get event by ID
 * @access  Public
 */
const getEventById = async (req, res, next) => {
  try {
    const event = await Event.findById(req.params.id)
      .populate('organizerId', 'name email department');

    if (!event) {
      return res.status(404).json({
        success: false,
        message: 'Event not found'
      });
    }

    res.json({
      success: true,
      data: { event }
    });
  } catch (error) {
    next(error);
  }
};

```

PDF generator for tickets and certificates

```
/**
 * PDF Generator Utility - Movie Style Ticket
 */

const PDFDocument = require('pdfkit');
const { generateQRCodeBuffer } = require('./qrGenerator');

// Rounded rectangle helper
const rrect = (doc, x, y, w, h, r) => {
  doc
    .moveTo(x + r, y)
    .lineTo(x + w - r, y)
    .quadraticCurveTo(x + w, y, x + w, y + r)
    .lineTo(x + w, y + h - r)
    .quadraticCurveTo(x + w, y + h, x + w - r, y + h)
    .lineTo(x + r, y + h)
    .quadraticCurveTo(x, y + h, x, y + h - r)
    .lineTo(x, y + r)
    .quadraticCurveTo(x, y, x + r, y)
    .closePath();
  return doc;
};

const generateTicketPDF = async (ticket) => {
  return new Promise(async (resolve, reject) => {
    try {
      const W = 360;
```

```
28        const H = 680;
29        const PAD = 20;
30        const R = 20;
31
32        const doc = new PDFDocument({
33          size: [W, H],
34          margin: 0,
35        });
36
37        const chunks = [];
38        doc.on('data', (c) => chunks.push(c));
39        doc.on('end', () => resolve(Buffer.concat(chunks)));
40        doc.on('error', reject);
41
42        // ■■ DATA ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
43        const event = ticket?.eventId || {};
44        const user = ticket?.userId || {};
45
46        const eventName = event?.title || 'EVENT NAME';
47        const attendeeName = user?.name || 'YOUR NAME';
48        const rollNumber = user?.rollNumber || '';
49        const department = user?.department || '';
50        const venue = event?.venue || '';
51        const code = ticket?.code || ticket?.ticketId || 'CODE';
52
53        const eventDate = new Date(event?.date);
54        const dateStr = isNaN(eventDate.getTime())
55          ? '—'
56          : eventDate.toLocaleDateString('en-US', {
57              weekday: 'short',
58              month: 'short',
59              day: 'numeric',
60              year: 'numeric',
61            });
62
63        const timeStr = event?.time || '—';
64
65        const verificationBase = process.env.TICKET_VERIFICATION_URL_BASE;
66        const qrValue = verificationBase
67          ? `${verificationBase}?ticketId=${encodeURIComponent(code)}`
68          : code;
69
70        // ■■ COLORS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
71        const BG = '#1c1c1e';
72        const WHITE = '#ffffff';
73        const MUTED = '#8e8e93';
74        const DIVIDER = '#3a3a3c';
75
76        // ■■ BACKGROUND ■■■■■■■■■■■■■■■■■■■■■■■■■
77        rrect(doc, 0, 0, W, H, R).fillColor(BG).fill();
78
79        // ■■ HEADER ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
80        doc.rect(0, 0, W, 70).fill('#111');
81
82        doc.fillColor(WHITE)
83          .font('Helvetica-Bold')
84          .fontSize(14)
85          .text('UNI EVENT PASS', PAD, 25);
86
87        doc.fillColor(MUTED)
88          .fontSize(10)
89          .text('Admit One', PAD, 45);
90
91        // ■■ EVENT TITLE ■■■■■■■■■■■■■■■■■■■■■■■■
92        let curY = 90;
93
94        doc.fillColor(WHITE)
95          .font('Helvetica-Bold')
96          .fontSize(20)
97          .text(eventName, PAD, curY, {
98            width: W - PAD * 2,
99          });
100
101       curY += doc.heightOfString(eventName, {
102         width: W - PAD * 2,
103       }) + 10;
104
105       // ■■ QR + SIDE INFO ■■■■■■■■■■■■■■■■■■■■
```

```
106         const qrSize = 120;
107         const qrX = W - qrSize - PAD;
108         const qrY = curY;
109
110         rrect(doc, qrX, qrY, qrSize, qrSize, 10).fillColor(WHITE).fill();
111
112         const qrBuffer = await generateQRCodeBuffer(qrValue);
113         doc.image(qrBuffer, qrX + 8, qrY + 8, {
114           width: qrSize - 16,
115           height: qrSize - 16,
116         });
117
118         let infoY = curY;
119
120         const info = [
121           { label: 'DATE', value: dateStr },
122           { label: 'TIME', value: timeStr },
123           { label: 'VENUE', value: venue || '—' },
124         ];
125
126         info.forEach((item) => {
127           doc.fillColor(MUTED)
128             .fontSize(8)
129             .text(item.label, PAD, infoY);
130
131           doc.fillColor(WHITE)
132             .font('Helvetica-Bold')
133             .fontSize(11)
134             .text(item.value, PAD, infoY + 10);
135
136           infoY += 28;
137         });
138
139         curY = qrY + qrSize + 20;
140
141         // ■■ TEAR LINE ■■■■■■■■■■■■■■■■■■■■■■■■■■
142         doc.circle(0, curY, 10).fill(BG);
143         doc.circle(W, curY, 10).fill(BG);
144
145         doc.strokeColor(DIVIDER)
146           .dash(6, { space: 4 })
147           .moveTo(PAD, curY)
148           .lineTo(W - PAD, curY)
149           .stroke()
150           .undash();
151
152         curY += 20;
153
154         // ■■ USER INFO ■■■■■■■■■■■■■■■■■■■■■■■■■■
155         doc.fillColor(MUTED)
156           .fontSize(8)
157           .text('ATTENDEE', PAD, curY);
158
159         doc.fillColor(WHITE)
160           .font('Helvetica-Bold')
161           .fontSize(14)
162           .text(attendeeName, PAD, curY + 12);
163
164         curY += 40;
165
166         // ■■ EXTRA INFO ■■■■■■■■■■■■■■■■■■■■■■■■■
167         const extras = [
168           { label: 'ROLL NO', value: rollNumber },
169           { label: 'DEPT', value: department },
170         ].filter(e => e.value);
171
172         extras.forEach((item, i) => {
173           const x = PAD + (i * 150);
174
175           doc.fillColor(MUTED)
176             .fontSize(8)
177             .text(item.label, x, curY);
178
179           doc.fillColor(WHITE)
180             .font('Helvetica-Bold')
181             .fontSize(11)
182             .text(item.value, x, curY + 10);
183         });
```

```

      curY += 50;

      // ■■ BARCODE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
      const bW = W - PAD * 2;
      const bH = 50;

      rrect(doc, PAD, curY, bW, bH, 8).fillColor('#111').fill();

      let state = 0;
      for (const ch of String(code)) {
        state = (state * 31 + ch.charCodeAt(0)) >>> 0;
      }

      const next = () => {
        state ^= state << 13; state >>>= 0;
        state ^= state >>> 17; state >>>= 0;
        state ^= state << 5;  state >>>= 0;
        return state;
      };

      doc.fillColor(WHITE);
      let bx = PAD + 10;

      while (bx < PAD + bW - 10) {
        const r = next();
        const bw = 1 + (r % 3);

        if (r % 2) {
          doc.rect(bx, curY + 8, bw, bH - 16).fill();
        }

        bx += bw;
      }

      curY += bH + 8;

      // ■■ CODE TEXT ■■■■■■■■■■■■■■■■■■■■■■■■■■
      doc.fillColor(MUTED)
        .fontSize(9)
        .text(code, 0, curY, {
          width: W,
          align: 'center',
          characterSpacing: 2,
        });

      curY += 20;

      // ■■ FOOTER ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
      doc.fillColor(DIVIDER)
        .fontSize(7)
        .text(
          'Valid for one person • Non-transferable • UniEvent',
          0,
          curY,
          { align: 'center', width: W }
        );

      doc.end();
    } catch (err) {
      reject(err);
    }
  });
};


/**
 * Generate a professional certificate PDF — A4 Landscape (unchanged)
 */
const generateCertificatePDF = async (certificate) => {
  return new Promise(async (resolve, reject) => {
    try {
      const doc = new PDFDocument({ size: 'A4', layout: 'landscape', margin: 0 });

      const chunks = [];
      doc.on('data', chunk => chunks.push(chunk));
      doc.on('end', () => resolve(Buffer.concat(chunks)));
```

Representative Frontend Source Code

Landing page

```
import Link from 'next/link';
import { Calendar, Ticket, Users, Shield } from 'lucide-react';
import { Button } from '@/components/ui';

export default function Home() {
  return (
    <div className="min-h-screen bg-neutral-50">
      {/* Navigation */}
      <nav className="bg-white/90 backdrop-blur-sm border-b border-neutral-100 sticky top-0 z-50">
        <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
          <div className="flex justify-between items-center h-16">
            <div className="flex items-center space-x-2.5">
              <Ticket className="h-7 w-7 text-neutral-700" />
              <span className="text-xl font-semibold text-neutral-900">UniEvent</span>
            </div>
            <div className="flex items-center space-x-3">
              <Link
                href="/login"
                className="text-neutral-600 hover:text-neutral-900 px-4 py-2 text-sm font-medium transition-c
              >
                Login
              </Link>
              <Link href="/signup">
                <Button>Sign Up</Button>
              </Link>
            </div>
          </div>
        </div>
      </nav>

      {/* Hero Section */}
      <section className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-24 sm:py-32">
        <div className="text-center">
          <h1 className="text-5xl sm:text-6xl lg:text-7xl font-semibold text-neutral-900 mb-6 tracking-tight"
            University Events,
            <br />
            <span className="text-neutral-600">Simplified</span>
          </h1>
          <p className="text-lg sm:text-xl text-neutral-600 mb-10 max-w-2xl mx-auto leading-relaxed">
            Discover events, get digital tickets, and earn certificates.
            A clean, simple platform for students and organizers.
          </p>
          <div className="flex flex-col sm:flex-row gap-3 justify-center">
            <Link href="/signup?role=student">
              <Button className="text-base px-8 py-3 w-full sm:w-auto">
                Join as Student
              </Button>
            </Link>
            <Link href="/signup?role=organizer">
              <Button variant="secondary" className="text-base px-8 py-3 w-full sm:w-auto">
                Register as Organizer
              </Button>
            </Link>
          </div>
        </div>
      </section>

      {/* Features Section */}
      <section className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-20">
        <h2 className="text-3xl font-semibold text-center text-neutral-900 mb-16 tracking-tight">
          Everything You Need
        </h2>
        <div className="grid md:grid-cols-2 lg:grid-cols-4 gap-6">
          <FeatureCard
            icon={<Calendar className="h-7 w-7" />}
            title="Event Management"
            description="Create and manage events with seat limits, schedules, and venue details."
          />
          <FeatureCard
            icon={<Ticket className="h-7 w-7" />}
            title="Digital Tickets"
            description="Generate QR-coded tickets instantly. Download as PDF for offline use."
```

```
          />
          <FeatureCard
            icon={<Shield className="h-7 w-7" />}
            title="Secure Entry"
            description="Scan QR codes for instant verification. Prevent duplicate entries."
          />
          <FeatureCard
            icon={<Users className="h-7 w-7" />}
            title="Certificates"
            description="Generate professional certificates for workshop attendees automatically."
          />
        </div>
      </section>

      {/* Footer */}
      <footer className="bg-neutral-900 text-neutral-400 py-10 mt-20">
        <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 text-center">
          <p className="text-sm">© 2026 UniEvent. University Event Management System.</p>
        </div>
      </footer>
    </div>
  );
}

function FeatureCard({
  icon,
  title,
  description,
}: {
  icon: React.ReactNode;
  title: string;
  description: string;
}) {
  return (
    <div className="bg-white rounded-2xl border border-neutral-100 p-8 text-center transition-all hover:borde
      <div className="inline-flex items-center justify-center w-12 h-12 rounded-xl bg-neutral-100 text-neutra
        {icon}
      </div>
      <h3 className="text-lg font-semibold text-neutral-900 mb-2.5">{title}</h3>
      <p className="text-neutral-600 text-sm leading-relaxed">{description}</p>
    </div>
  );
}
```

Student dashboard

```
'use client';

import { useEffect, useMemo, useState } from 'react';
import { useRouter } from 'next/navigation';
import { useAuth } from '@/context/AuthContext';
import { useToast } from '@/context/ToastContext';
import { useApi } from '@/hooks/useApi';
import { eventsAPI, ticketsAPI, certificatesAPI } from '@/lib/api';
import { formatDate, getStatusColor, downloadBlob, getImageUrl } from '@/lib/utils';
import {
  Calendar, Ticket, Award, LogOut,
  MapPin, Clock, Users, Download, FileText, Eye,
  ChevronRight, UserCircle, Search, Filter
} from 'lucide-react';
import type { Event, Ticket as TicketType, Certificate } from '@/types';
import CertificatePreviewModal from '@/components/CertificatePreviewModal';
import TicketPreviewModal from '@/components/TicketPreviewModal';

const DEPARTMENT_OPTIONS = [
  'Computer Science',
  'Electrical Engineering',
  'Mechanical Engineering',
  'Civil Engineering',
  'Electronics',
  'Information Technology',
  'MBA',
  'Other',
];

export default function StudentDashboard() {
```

```
31    const router = useRouter();
32    const { user, logout } = useAuth();
33    const toast = useToast();
34    const api = useApi();
35    const [events, setEvents] = useState<Event[]>([]);
36    const [ticketsAll, setTicketsAll] = useState<TicketType[]>([]);
37    const [ticketsPageItems, setTicketsPageItems] = useState<TicketType[]>([]);
38    const [certificates, setCertificates] = useState<Certificate[]>([]);
39    const [isLoading, setIsLoading] = useState(true);
40    const [activeTab, setActiveTab] = useState<'events' | 'tickets' | 'certificates'>('events');
41
42    // Pagination
43    const EVENTS_LIMIT = 6;
44    const TICKETS_LIMIT = 6;
45
46    const [eventsPage, setEventsPage] = useState(1);
47    const [eventsTotalPages, setEventsTotalPages] = useState(1);
48    const [eventsTotal, setEventsTotal] = useState(0);
49
50    const [ticketsPage, setTicketsPage] = useState(1);
51    const [ticketsTotalPages, setTicketsTotalPages] = useState(1);
52    const [ticketsTotal, setTicketsTotal] = useState(0);
53
54    // Event search/filter
55    const [eventSearchInput, setEventSearchInput] = useState('');
56    const [eventSearch, setEventSearch] = useState('');
57    const [eventDepartment, setEventDepartment] = useState('');
58    const [eventType, setEventType] = useState<'all' | 'public' | 'departmental'>('all');
59    const [dateFrom, setDateFrom] = useState('');
60    const [dateTo, setDateTo] = useState('');
61    const [ticketFilter, setTicketFilter] = useState<'all' | 'unused' | 'used' | 'cancelled'>('all');
62    const [previewCertificate, setPreviewCertificate] = useState<Certificate | null>(null);
63    const [previewTicket, setPreviewTicket] = useState<TicketType | null>(null);
64    const [registeringEventId, setRegisteringEventId] = useState<string | null>(null);
65
66    // Read initial page state from query string (client-side) to avoid Next.js useSearchParams SSR issues
67    useEffect(() => {
68      const params = new URLSearchParams(window.location.search);
69      const ep = parseInt(params.get('eventsPage') || '1', 10);
70      const tp = parseInt(params.get('ticketsPage') || '1', 10);
71      if (!isNaN(ep) && ep > 0) setEventsPage(ep);
72      if (!isNaN(tp) && tp > 0) setTicketsPage(tp);
73    }, []);
74
75    // Debounce event search input
76    useEffect(() => {
77      const t = setTimeout(() => setEventSearch(eventSearchInput.trim()), 350);
78      return () => clearTimeout(t);
79    }, [eventSearchInput]);
80
81    // Keep pages in query string
82    useEffect(() => {
83      const url = new URL(window.location.href);
84      url.searchParams.set('eventsPage', String(eventsPage));
85      url.searchParams.set('ticketsPage', String(ticketsPage));
86      window.history.replaceState(null, '', url.toString());
87    }, [eventsPage, ticketsPage]);
88
89    const fetchEvents = async () => {
90      const params: any = {
91        upcoming: 'true',
92        page: eventsPage,
93        limit: EVENTS_LIMIT,
94      };
95
96      if (eventSearch) params.search = eventSearch;
97      if (eventDepartment) params.department = eventDepartment;
98      if (eventType !== 'all') params.type = eventType;
99      if (dateFrom) params.dateFrom = dateFrom;
100     if (dateTo) params.dateTo = dateTo;
101
102     try {
103       const eventsRes = await eventsAPI.getAll(params);
104       setEvents(eventsRes.data.data.events);
105       setEventsTotal(eventsRes.data.total || 0);
106       setEventsTotalPages(eventsRes.data.totalPages || 1);
107     } catch (error: any) {
108       toast.error(error.response?.data?.message || 'Failed to load events');
```

```
109       }
110     };
111
112     const fetchTicketsAll = async () => {
113       try {
114         const ticketsRes = await ticketsAPI.getMyTickets();
115         setTicketsAll(ticketsRes.data.data.tickets);
116       } catch (error: any) {
117         toast.error(error.response?.data?.message || 'Failed to load tickets');
118       }
119     };
120
121     const fetchTicketsPage = async () => {
122       try {
123         const ticketsRes = await ticketsAPI.getAll({ page: ticketsPage, limit: TICKETS_LIMIT });
124         setTicketsPageItems(ticketsRes.data.data.tickets);
125         setTicketsTotal(ticketsRes.data.total || 0);
126         setTicketsTotalPages(ticketsRes.data.totalPages || 1);
127       } catch (error: any) {
128         toast.error(error.response?.data?.message || 'Failed to load tickets');
129       }
130     };
131
132     const fetchCertificates = async () => {
133       try {
134         const certificatesRes = await certificatesAPI.getMyCertificates();
135         setCertificates(certificatesRes.data.data.certificates || []);
136       } catch (error: any) {
137         toast.error(error.response?.data?.message || 'Failed to load certificates');
138       }
139     };
140
141     useEffect(() => {
142       // Initial load
143       (async () => {
144         setIsLoading(true);
145         await Promise.all([fetchEvents(), fetchTicketsAll(), fetchTicketsPage(), fetchCertificates()]);
146         setIsLoading(false);
147       })();
148       // eslint-disable-next-line react-hooks/exhaustive-deps
149     }, []);
150
151     useEffect(() => {
152       fetchEvents();
153       // eslint-disable-next-line react-hooks/exhaustive-deps
154     }, [eventsPage, eventSearch, eventDepartment, eventType, dateFrom, dateTo]);
155
156     useEffect(() => {
157       fetchTicketsPage();
158       // eslint-disable-next-line react-hooks/exhaustive-deps
159     }, [ticketsPage]);
160
161     const handleRegister = async (eventId: string) => {
162       setRegisteringEventId(eventId);
163
164       await api.run(() => ticketsAPI.register(eventId), {
165         successMessage: 'Successfully registered for the event',
166         errorMessage: (err) => err?.response?.data?.message || 'Registration failed',
167         onSuccess: async () => {
168           await Promise.all([fetchEvents(), fetchTicketsAll(), fetchTicketsPage()]);
169         },
170       });
171
172       setRegisteringEventId(null);
173     };
174
175     const handleDownloadTicket = async (ticketId: string) => {
176       try {
177         const response = await ticketsAPI.download(ticketId);
178         downloadBlob(response.data, `ticket-${ticketId}.pdf`);
179         toast.success('Ticket downloaded');
180       } catch (error) {
181         toast.error('Failed to download ticket');
182       }
183     };
184
185     const handleDownloadCertificate = async (certificateId: string) => {
186       try {
```

```
      const response = await certificatesAPI.download(certificateId);
      downloadBlob(response.data, `certificate-${certificateId}.pdf`);
      toast.success('Certificate downloaded');
    } catch (error) {
      toast.error('Failed to download certificate');
    }
  };

  const isRegistered = (eventId: string) => {
    return ticketsAll.some(t => {
      // `t.eventId` can be a string, a populated event object, or null (e.g. if the event was deleted).
      const eId =
        typeof t.eventId === 'string'
          ? t.eventId
          : t.eventId?._id;

      return !!eId && eId === eventId;
    });
  };

  const sortedAndFilteredTickets = useMemo(() => {
    const filtered = ticketFilter === 'all'
      ? ticketsPageItems
      : ticketsPageItems.filter(t => t.status === ticketFilter);

    // Stable sort: keep original index as final tie-breaker
    const withIndex = filtered.map((t, idx) => ({ t, idx }));

    const getEventDateMs = (ticket: TicketType): number => {
      const event = typeof ticket.eventId === 'object' ? ticket.eventId : null;
      const date = event?.date;
      const ms = date ? new Date(date).getTime() : Number.POSITIVE_INFINITY;
      return Number.isFinite(ms) ? ms : Number.POSITIVE_INFINITY;
    };

    withIndex.sort((a, b) => {
      const aMs = getEventDateMs(a.t);
      const bMs = getEventDateMs(b.t);
      if (aMs !== bMs) return aMs - bMs;
      return a.idx - b.idx;
    });

    return withIndex.map(x => x.t);
  }, [ticketsPageItems, ticketFilter]);

  return (
    <div className="min-h-screen bg-neutral-50">
      {/* Header */}
      <header className="bg-white border-b border-neutral-100 sticky top-0 z-50">
        <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
          <div className="flex justify-between items-center h-16">
            <div className="flex items-center space-x-2.5">
              <Ticket className="h-7 w-7 text-neutral-700" />
              <span className="text-xl font-semibold text-neutral-900">UniEvent</span>
            </div>
            <div className="flex items-center space-x-4">
              <span className="text-sm text-neutral-600">
                <strong className="text-neutral-900">{user?.name}</strong>
              </span>
              <button
                onClick={() => router.push('/dashboard/student/profile')}
                className="p-2 rounded-xl hover:bg-neutral-100 text-neutral-700"
                aria-label="Profile"
                title="Profile"
              >
                <UserCircle className="h-5 w-5" />
              </button>
              <button
                onClick={logout}
                className="flex items-center gap-2 text-neutral-600 hover:text-neutral-900 transition-colors"
              >
                <LogOut className="h-4 w-4" />
                Logout
              </button>
            </div>
          </div>
        </div>
      </header>
```

```
265
266         <main className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-10">
267           {/* Stat Cards - Clickable */}
268           <div className="grid grid-cols-1 sm:grid-cols-3 gap-4 mb-10">
269             <StatCard
270               icon={<Calendar className="h-6 w-6" />}
271               label="Upcoming Events"
272               value={eventsTotal || events.length}
273               onClick={() => setActiveTab('events')}
274               isActive={activeTab === 'events'}
275             />
276             <StatCard
277               icon={<Ticket className="h-6 w-6" />}
278               label="My Tickets"
279               value={ticketsTotal || ticketsAll.length}
280               onClick={() => setActiveTab('tickets')}
281               isActive={activeTab === 'tickets'}
282             />
283             <StatCard
284               icon={<Award className="h-6 w-6" />}
285               label="Certificates"
286               value={certificates.length}
287               onClick={() => setActiveTab('certificates')}
288               isActive={activeTab === 'certificates'}
289             />
290           </div>
291
292           {/* Tabs */}
293           <div className="flex flex-wrap gap-3 mb-8">
294             <button
295               onClick={() => setActiveTab('events')}
296               className={`px-5 py-2.5 rounded-xl font-medium transition-all ${
297                 activeTab === 'events'
298                   ? 'bg-neutral-900 text-white shadow-sm'
299                   : 'bg-white text-neutral-600 hover:bg-neutral-100 border border-neutral-200'
300               }`}
301             >
302               Browse Events
303             </button>
304             <button
305               onClick={() => setActiveTab('tickets')}
306               className={`px-5 py-2.5 rounded-xl font-medium transition-all ${
307                 activeTab === 'tickets'
308                   ? 'bg-neutral-900 text-white shadow-sm'
309                   : 'bg-white text-neutral-600 hover:bg-neutral-100 border border-neutral-200'
310               }`}
311             >
312               My Tickets
313             </button>
314             <button
315               onClick={() => setActiveTab('certificates')}
316               className={`px-5 py-2.5 rounded-xl font-medium transition-all ${
317                 activeTab === 'certificates'
318                   ? 'bg-neutral-900 text-white shadow-sm'
319                   : 'bg-white text-neutral-600 hover:bg-neutral-100 border border-neutral-200'
320               }`}
321             >
322               Certificates
323             </button>
324           </div>
325
326           {/* Content */}
327           {isLoading ? (
328             <div className="grid md:grid-cols-2 lg:grid-cols-3 gap-5">
329               {[1, 2, 3, 4, 5, 6].map((i) => (
330                 <CardSkeleton key={i} />
331               ))}
332             </div>
333           ) : activeTab === 'events' ? (
334             <div className="grid md:grid-cols-2 lg:grid-cols-3 gap-5">
335               {/* Search & filters */}
336               <div className="md:col-span-2 lg:col-span-3 mb-2">
337                 <div className="flex flex-col sm:flex-row gap-3">
338                   <div className="relative flex-1">
339                     <Search className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-neutral-400" />
340                     <input
341                       value={eventSearchInput}
342                       onChange={(e) => {
```

```
                        setEventsPage(1);
                        setEventSearchInput(e.target.value);
                      }}
                      placeholder="Search events…"
                      className="w-full pl-10 pr-3 py-2.5 bg-white border border-neutral-200 rounded-xl text-sm
                    />
                  </div>

```

Organizer dashboard

```
'use client';

import { useEffect, useState, useRef } from 'react';
import { useRouter } from 'next/navigation';
import { useAuth } from '@/context/AuthContext';
import { useToast } from '@/context/ToastContext';
import { useApi } from '@/hooks/useApi';
import { eventsAPI, ticketsAPI } from '@/lib/api';
import { formatDate, getStatusColor, getImageUrl, isValidImageType } from '@/lib/utils';
import {
  Calendar, Ticket, Users, LogOut, Plus,
  QrCode, CheckCircle, XCircle,
  BarChart3, Eye, Camera, Keyboard, Award, Upload, X,
  Filter, ArrowUpDown, Trash2, Pencil, ChevronDown, MapPin, UserCircle
} from 'lucide-react';
import type { Event, EventStatus } from '@/types';
import CameraScan from '@/components/CameraScan';
import { EditEventModal } from '@/components/EditEventModal';
import { ConfirmModal } from '@/components/ui/ConfirmModal';

export default function OrganizerDashboard() {
  const router = useRouter();
  const { user, logout } = useAuth();
  const toast = useToast();
  const api = useApi();
  const [events, setEvents] = useState<Event[]>([]);
  const [isLoading, setIsLoading] = useState(true);
  const [showCreateModal, setShowCreateModal] = useState(false);
  const [showScanModal, setShowScanModal] = useState(false);
  const [selectedScanEvent, setSelectedScanEvent] = useState<Event | null>(null);
  const [showEditModal, setShowEditModal] = useState(false);
  const [selectedEditEvent, setSelectedEditEvent] = useState<Event | null>(null);

  // Delete confirmation modal state (kept at page-level to avoid nested fixed-modal glitches)
  const [deleteCandidate, setDeleteCandidate] = useState<Event | null>(null);
  const [isDeletingEvent, setIsDeletingEvent] = useState(false);

  // Filter and sort state
  const [statusFilter, setStatusFilter] = useState<string>('all');
  const [sortBy, setSortBy] = useState<string>('newest');

  // Filter and sort events
  const filteredEvents = events
    .filter(event => statusFilter === 'all' || event.status === statusFilter)
    .sort((a, b) => {
      switch (sortBy) {
        case 'oldest':
          return new Date(a.createdAt || a.date).getTime() - new Date(b.createdAt || b.date).getTime();
        case 'date_asc':
          return new Date(a.date).getTime() - new Date(b.date).getTime();
        case 'date_desc':
          return new Date(b.date).getTime() - new Date(a.date).getTime();
        case 'newest':
        default:
          return new Date(b.createdAt || b.date).getTime() - new Date(a.createdAt || a.date).getTime();
      }
    });

  useEffect(() => {
    fetchEvents();
  }, []);

  const fetchEvents = async () => {
    try {
      const response = await eventsAPI.getAll();
```

```
      setEvents(response.data.data.events);
    } catch (error: any) {
      toast.error(error.response?.data?.message || 'Failed to load events');
    } finally {
      setIsLoading(false);
    }
  };

  const handleConfirmDelete = async () => {
    if (!deleteCandidate) return;

    const toDelete = deleteCandidate;
    const previous = events;

    setIsDeletingEvent(true);

    await api.run(() => eventsAPI.delete(toDelete._id), {
      optimisticUpdate: () => {
        // Optimistic UI: remove immediately
        setEvents((prev) => prev.filter((e) => e._id !== toDelete._id));
      },
      rollback: () => {
        setEvents(previous);
      },
      successMessage: 'Event deleted successfully',
      errorMessage: (err) => err?.response?.data?.message || 'Failed to delete event',
      onSuccess: () => {
        setDeleteCandidate(null);
      },
    });

    setIsDeletingEvent(false);
  };

  return (
    <div className="min-h-screen bg-neutral-50">
      {/* Header */}
      <header className="bg-white border-b border-neutral-100 sticky top-0 z-50">
        <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
          <div className="flex justify-between items-center h-16">
            <div className="flex items-center space-x-2.5">
              <Ticket className="h-7 w-7 text-neutral-700" />
              <span className="text-xl font-semibold text-neutral-900">UniEvent</span>
              <span className="text-sm bg-neutral-100 text-neutral-700 px-2.5 py-1 rounded-lg font-medium">
                Organizer
              </span>
            </div>
            <div className="flex items-center space-x-4">
              <span className="text-sm text-neutral-600">
                <strong className="text-neutral-900">{user?.name}</strong> • {user?.department}
              </span>
              <button
                onClick={() => router.push('/dashboard/organizer/profile')}
                className="p-2 rounded-xl hover:bg-neutral-100 text-neutral-700"
                aria-label="Profile"
                title="Profile"
              >
                <UserCircle className="h-5 w-5" />
              </button>
              <button
                onClick={logout}
                className="flex items-center gap-2 text-neutral-600 hover:text-neutral-900 transition-colors"
              >
                <LogOut className="h-4 w-4" />
                Logout
              </button>
            </div>
          </div>
        </div>
      </header>

      <main className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-10">
        {/* Stats */}
        <div className="grid grid-cols-1 md:grid-cols-4 gap-5 mb-10">
          <StatCard
            icon={<Calendar className="h-5 w-5" />}
            label="Total Events"
            value={events.length}
```

```
144             />
145             <StatCard
146               icon={<Users className="h-5 w-5" />}
147               label="Total Registrations"
148               value={events.reduce((sum, e) => sum + e.registeredCount, 0)}
149             />
150             <StatCard
151               icon={<Ticket className="h-5 w-5" />}
152               label="Published"
153               value={events.filter(e => e.status === 'published').length}
154             />
155             <StatCard
156               icon={<BarChart3 className="h-5 w-5" />}
157               label="Completed"
158               value={events.filter(e => e.status === 'completed').length}
159             />
160           </div>
161
162           {/* Header & Actions */}
163           <div className="flex flex-col sm:flex-row justify-between items-start sm:items-center gap-4 mb-6">
164             <h2 className="text-xl font-semibold text-neutral-900">My Events</h2>
165             <button
166               onClick={() => setShowCreateModal(true)}
167               className="w-full sm:w-auto px-4 py-2.5 bg-neutral-900 text-white rounded-xl font-medium hover:bg
168             >
169               <Plus className="h-4 w-4" />
170               Create Event
171             </button>
172           </div>
173
174           {/* Filter & Sort Controls */}
175           <div className="flex flex-col sm:flex-row gap-3 mb-6">
176             {/* Status Filter */}
177             <div className="relative">
178               <select
179                 value={statusFilter}
180                 onChange={(e) => setStatusFilter(e.target.value)}
181                 className="appearance-none w-full sm:w-auto pl-10 pr-10 py-2.5 bg-white border border-neutral-2
182               >
183                 <option value="all">All Status</option>
184                 <option value="draft">Draft</option>
185                 <option value="published">Published</option>
186                 <option value="ongoing">Ongoing</option>
187                 <option value="completed">Completed</option>
188               </select>
189               <Filter className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-neutral-400" />
190               <ChevronDown className="absolute right-3 top-1/2 -translate-y-1/2 h-4 w-4 text-neutral-400" />
191             </div>
192
193             {/* Sort Dropdown */}
194             <div className="relative">
195               <select
196                 value={sortBy}
197                 onChange={(e) => setSortBy(e.target.value)}
198                 className="appearance-none w-full sm:w-auto pl-10 pr-10 py-2.5 bg-white border border-neutral-2
199               >
200                 <option value="newest">Newest First</option>
201                 <option value="oldest">Oldest First</option>
202                 <option value="date_asc">Date (Ascending)</option>
203                 <option value="date_desc">Date (Descending)</option>
204               </select>
205               <ArrowUpDown className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-neutral-400" />
206               <ChevronDown className="absolute right-3 top-1/2 -translate-y-1/2 h-4 w-4 text-neutral-400" />
207             </div>
208
209             {/* Results count */}
210             <div className="flex items-center text-sm text-neutral-500 sm:ml-auto">
211               Showing {filteredEvents.length} of {events.length} events
212             </div>
213           </div>
214
215           {/* Events Grid */}
216           {isLoading ? (
217             <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-5">
218               {[1, 2, 3].map((i) => (
219                 <EventCardSkeleton key={i} />
220               ))}
221             </div>
```

```
222          ) : (
223            <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-5">
224              {filteredEvents.map((event) => (
225                <EventCard
226                  key={event._id}
227                  event={event}
228                  onScan={() => {
229                    setSelectedScanEvent(event);
230                    setShowScanModal(true);
231                  }}
232                  onEdit={() => {
233                    setSelectedEditEvent(event);
234                    setShowEditModal(true);
235                  }}
236                  onDelete={() => setDeleteCandidate(event)}
237                  onView={() => router.push(`/dashboard/organizer/events/${event._id}`)}
238                  isDeleting={isDeletingEvent && deleteCandidate?._id === event._id}
239                  onSetStatus={(eventId: string, status: EventStatus) => {
240                    setEvents((prev) => prev.map((e) => (e._id === eventId ? { ...e, status } : e)));
241                  }}
242                  onReplace={(eventId: string, patch: Partial<Event>) => {
243                    setEvents((prev) => prev.map((e) => (e._id === eventId ? { ...e, ...patch } : e)));
244                  }}
245                />
246              ))}
247              {filteredEvents.length === 0 && events.length > 0 && (
248                <div className="col-span-full bg-white rounded-2xl border border-neutral-100 p-12 text-center">
249                  <p className="text-neutral-500">No events match your filter.</p>
250                  <button
251                    onClick={() => setStatusFilter('all')}
252                    className="mt-4 px-4 py-2 bg-neutral-100 text-neutral-700 rounded-xl font-medium hover:bg-n
253                  >
254                    Clear Filter
255                  </button>
256                </div>
257              )}
258              {events.length === 0 && (
259                <div className="col-span-full bg-white rounded-2xl border border-neutral-100 p-12 text-center">
260                  <Calendar className="h-12 w-12 text-neutral-300 mx-auto mb-4" />
261                  <p className="text-neutral-500 mb-4">You haven't created any events yet.</p>
262                  <button
263                    onClick={() => setShowCreateModal(true)}
264                    className="px-4 py-2.5 bg-neutral-900 text-white rounded-xl font-medium hover:bg-neutral-80
265                  >
266                    Create Your First Event
267                  </button>
268                </div>
269              )}
270            </div>
271          )}
272        </main>
273
274        {/* Create Event Modal */}
275        {showCreateModal && (
276          <CreateEventModal
277            onClose={() => setShowCreateModal(false)}
278            onOptimisticCreate={(tempEvent: Event) => {
279              setEvents((prev) => [tempEvent, ...prev]);
280            }}
281            onCommitCreate={(tempId: string, realEvent: Event) => {
282              setEvents((prev) => prev.map((e) => (e._id === tempId ? realEvent : e)));
283            }}
284            onRollbackCreate={(tempId: string) => {
285              setEvents((prev) => prev.filter((e) => e._id !== tempId));
286            }}
287            onSuccess={() => {
288              setShowCreateModal(false);
289            }}
290          />
291        )}
292
293        {/* QR Scan Modal */}
294        {showScanModal && selectedScanEvent && (
295          <QRScanModal
296            event={selectedScanEvent}
297            onClose={() => setShowScanModal(false)}
298          />
299        )}
```

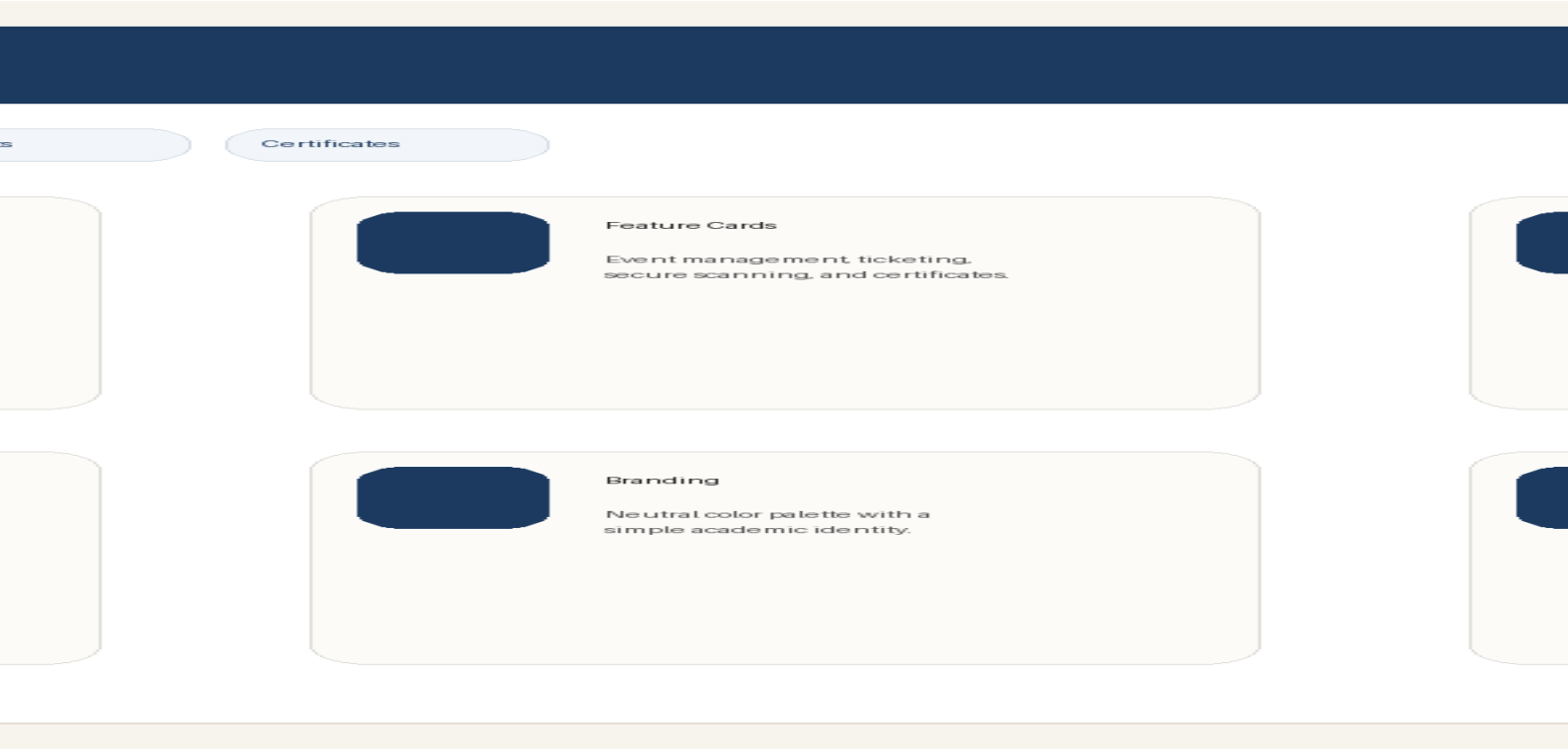
```

      {/* Edit Event Modal */}
      {showEditModal && selectedEditEvent && (
        <EditEventModal
          event={selectedEditEvent}
          onClose={() => setShowEditModal(false)}
          onSuccess={() => {
            setShowEditModal(false);
            fetchEvents();
          }}
        />
      )}

      {/* Delete Confirmation Modal */}
      <ConfirmModal
        isOpen={!!deleteCandidate}
        onClose={() => {
          if (isDeletingEvent) return;
          setDeleteCandidate(null);
        }}
        onConfirm={handleConfirmDelete}
        title="Delete event"
        message="Are you sure you want to delete this event?"
        confirmText="Delete"
        cancelText="Cancel"
        variant="danger"
        isLoading={isDeletingEvent}
      />
    </div>
  );
}

function StatCard({ icon, label, value }: any) {
  return (
    <div className="bg-white rounded-2xl border border-neutral-100 p-6 flex items-center gap-4">
      <div className="p-3 rounded-xl bg-neutral-100 text-neutral-700">
        {icon}
      </div>
      <div>
        <p className="text-2xl font-semibold text-neutral-900">{value}</p>
        <p className="text-sm text-neutral-600">{label}</p>
      </div>
    </div>
  );
}

// Skeleton loader for event cards
function EventCardSkeleton() {
  return (
    <div className="bg-white rounded-2xl border border-neutral-100 overflow-hidden animate-pulse">
      <div className="h-40 w-full bg-neutral-200" />
      <div className="p-5">
        <div className="h-5 bg-neutral-200 rounded w-3/4 mb-3" />
        <div className="h-4 bg-neutral-100 rounded w-1/2 mb-2" />
        <div className="h-4 bg-neutral-100 rounded w-2/3 mb-4" />
        <div className="flex gap-2">
          <div className="h-8 bg-neutral-200 rounded-lg w-20" />
          <div className="h-8 bg-neutral-100 rounded-lg w-24" />
        </div>
      </div>
    </div>
  );
}

function EventCard({ event, onScan, onEdit, onDelete, onView, isDeleting, onSetStatus, onReplace }: any) {
  const toast = useToast();
  const api = useApi();
  const [isUpdating, setIsUpdating] = useState(false);

  const handleStatusChange = async (newStatus: EventStatus) => {
    const prevStatus: EventStatus = event.status;

    setIsUpdating(true);

    await api.run(() => eventsAPI.update(event._id, { status: newStatus }), {
      optimisticUpdate: () => {
        onSetStatus?.(event._id, newStatus);
      },
```

```
      rollback: () => {
        onSetStatus?.(event._id, prevStatus);
      },
      successMessage: (response: any) => {
        const certResult = response.data?.data?.certificates;

        if (newStatus === 'published') return 'Event published';
        if (newStatus === 'completed') {
          if (certResult?.generated > 0) {
            return `Event completed — ${certResult.generated} certificates generated`;
          }
          return 'Event marked as completed';
        }

        return 'Event status updated';
      },
      errorMessage: (err) => err?.response?.data?.message || 'Failed to update event status',
      onSuccess: (response: any) => {
        const updatedEvent = response.data?.data?.event;
        if (updatedEvent) {
          onReplace?.(event._id, updatedEvent);
        }
      },
    });

    setIsUpdating(false);
  };


  const bannerUrl = getImageUrl(event.bannerUrl);

  return (
    <div className="bg-white rounded-2xl border border-neutral-100 overflow-hidden max-w-sm w-full hover:shad
      {/* Event Banner */}
      <div className="h-36 w-full overflow-hidden relative">
        {bannerUrl ? (
          <img
            src={bannerUrl}
            alt={event.title}
            className="w-full h-full object-cover"
          />
        ) : (
          <div className="w-full h-full bg-gradient-to-br from-neutral-200 to-neutral-300 flex items-center j
```

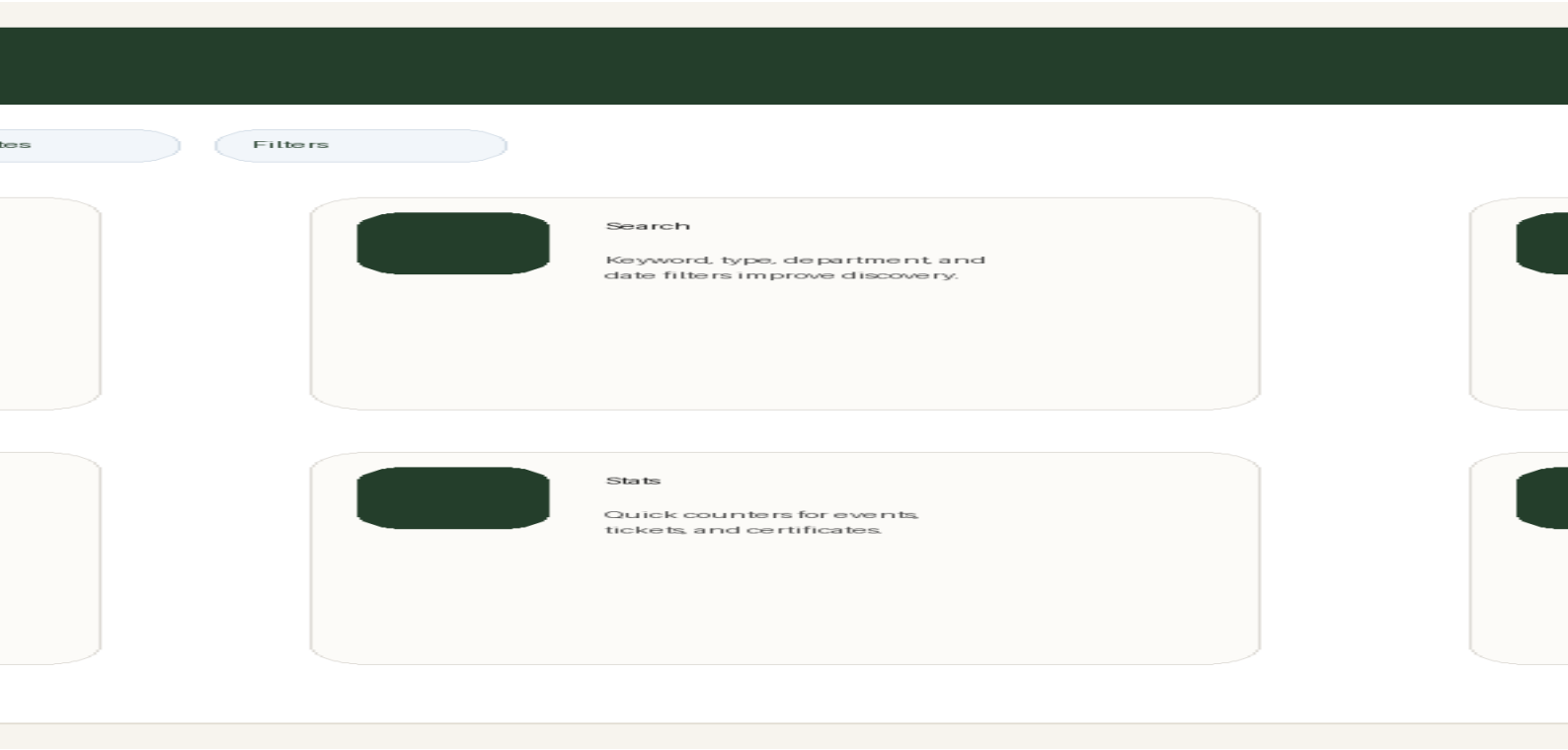
Screenshots of UI

This chapter consolidates the representative screens used during analysis and implementation review. The screenshots correspond to implemented pages and document the visual flows that would typically be presented during internal and external viva-voce demonstrations.

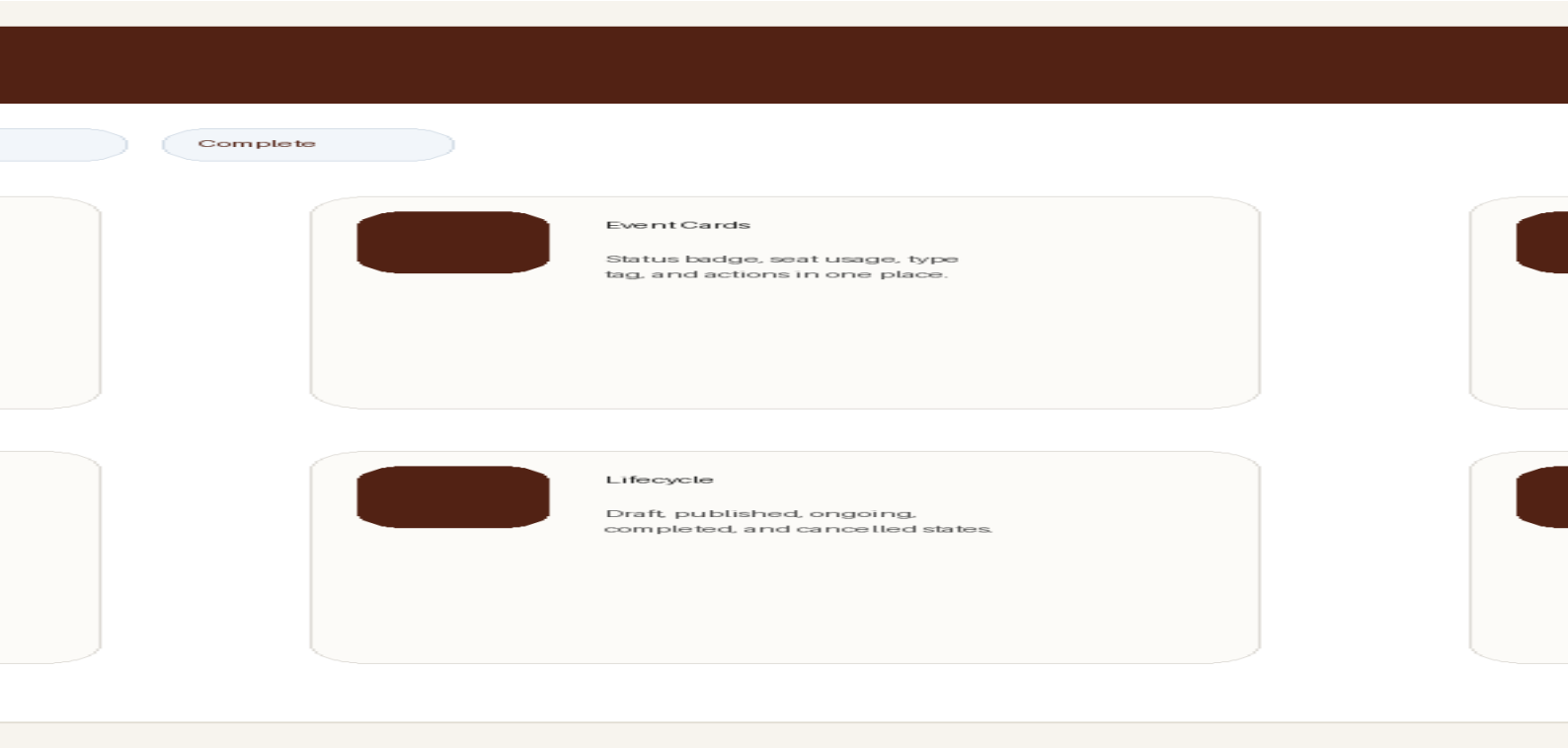
The landing page communicates the purpose of the system immediately. The student dashboard demonstrates the discover-register-download workflow, while the organizer dashboard shows administration and verification responsibilities. Ticket and certificate previews confirm that the project produces formal downloadable outputs rather than merely storing text data.



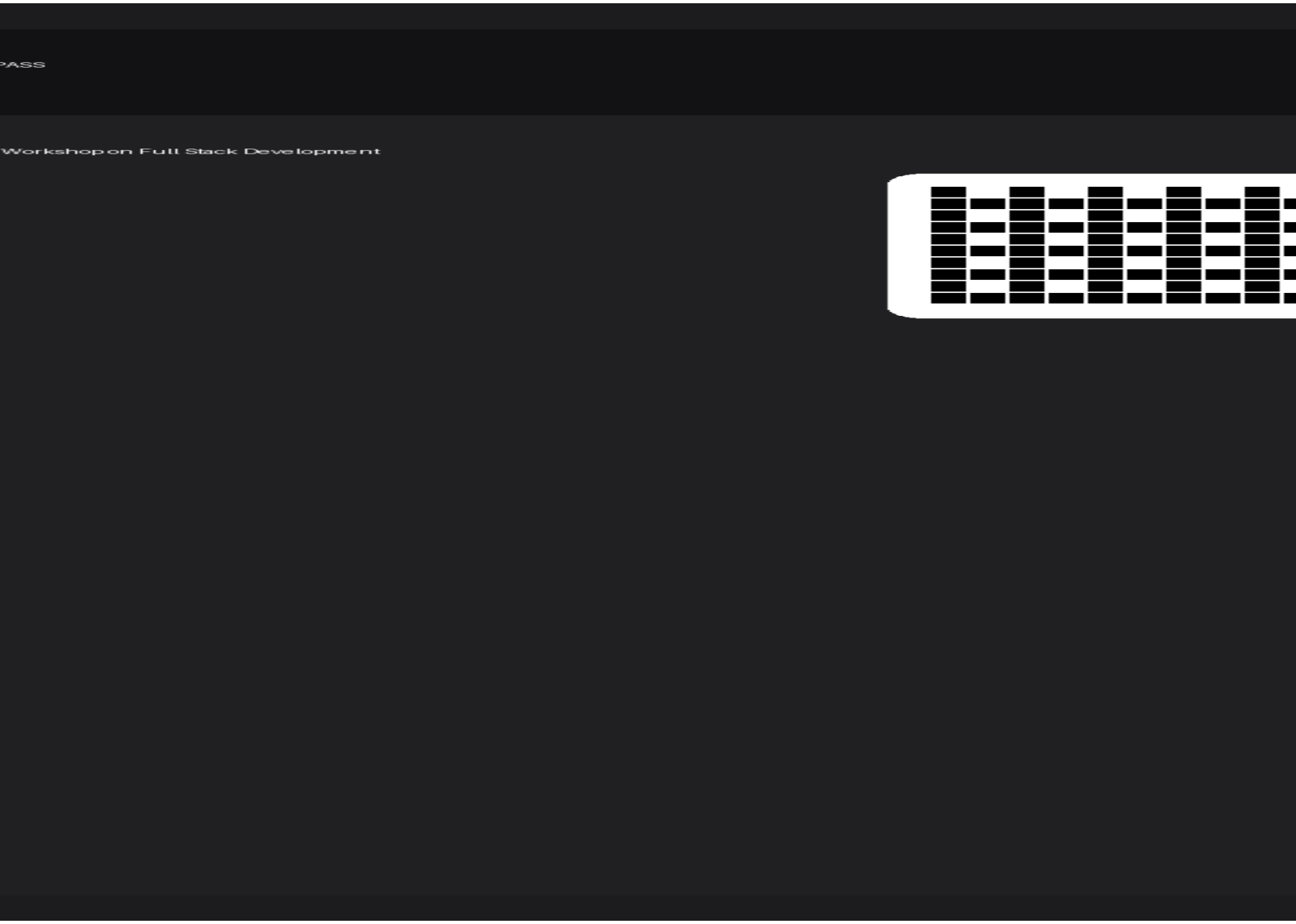
Screenshot 1: Landing page.



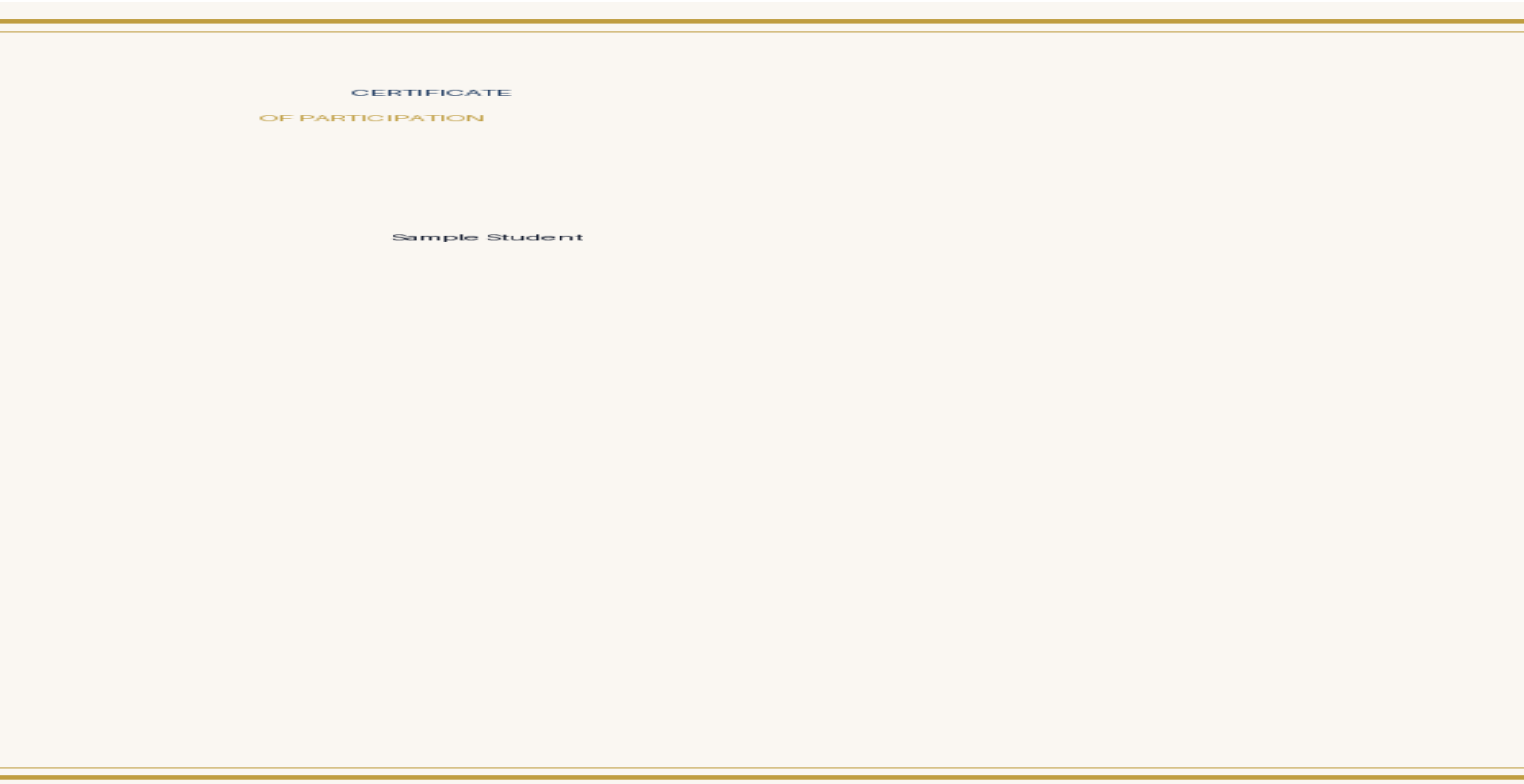
Screenshot 2: Student dashboard.



Screenshot 3: Organizer dashboard.



Screenshot 4: Ticket preview.



Screenshot 5: Certificate preview.

Testing

Testing for UniEvent was performed with a combination of manual functional checks, negative validation scenarios, API contract review, and code-level reasoning against the implemented business rules. The project structure also includes Jest, Supertest, and mongodb-memory-server in the backend dependencies, indicating a path for automated API-level testing. Even when viva timelines require extensive manual demonstration, the existing architecture is suitable for repeatable automated tests.

The main testing focus areas were authentication correctness, event CRUD integrity, registration eligibility, duplicate registration prevention, ticket verification rules, attendance recording, and artifact downloads. Testing also examined role boundaries because one of the most critical project requirements is separation between student and organizer actions.

TC ID	Scenario	Expected Result
TC-01	Student signup with valid data	Account created successfully and JWT is issued after login.
TC-02	Signup with invalid email	Validation error returned and record is not created.
TC-03	Student login with wrong password	Authentication fails with appropriate error message.
TC-04	Organizer creates draft event	Event is stored with draft status and organizer ownership.
TC-05	Organizer publishes event	Event status changes to published and becomes visible to stud
TC-06	Student registers when seats are available	Ticket is created, QR is generated, and count increments.
TC-07	Student tries duplicate registration	System rejects the request due to existing ticket.
TC-08	Student attempts departmental event of another department	Access is denied with explanatory message.
TC-09	Organizer scans valid unused ticket	Verification succeeds, ticket becomes used, attendance is crea
TC-10	Organizer scans already used ticket	System rejects verification and reports ALREADY_USED.
TC-11	Ticket scanned for wrong event	System rejects verification and reports WRONG_EVENT.
TC-12	Organizer completes certificate-enabled event	Certificates are generated for valid attendees.
TC-13	Student downloads ticket PDF	PDF file is returned in correct format.
TC-14	Student downloads certificate PDF	Certificate PDF is returned for issued certificate.
TC-15	Unauthorized user accesses organizer-only route	Request is blocked with authorization error.

Test Cases & Test Criteria

The acceptance criteria for the project were aligned with both academic requirements and practical user flows. An event should never oversubscribe past its seat limit. Each student should have at most one ticket per event. A ticket should be acceptable only once, and the QR payload should reveal no personal data on its own. Departmental eligibility should be enforced in the business layer rather than left to organizer memory. The system should generate professional PDFs for both tickets and certificates.

Usability criteria included a clear landing page, understandable dashboard actions, short feedback messages, and visible status indicators. Performance criteria focused on quick registration responses, low-friction event browsing, and reasonably fast ticket verification at entry time. Security criteria included authenticated access, route protection, password hashing, and input validation.

Criterion	Measure	Pass Condition
Functional completeness	Coverage of listed modules	Authentication, events, tickets, attendance, certificates all c
Data integrity	Registration and status transitions	No duplicate tickets, no negative seat counts, no invalid tic
Security	Role and credential protection	Unauthorized operations blocked; passwords hashed; auth
Usability	Navigation clarity	Users can complete the primary task flow without external
Output quality	PDF and QR artifacts	Generated ticket and certificate files are readable and form

Implementation & Evaluation of Project

The project has been implemented as a realistic full-stack web application instead of a static prototype. The backend package includes runtime scripts for development and production, and the frontend package includes build and start scripts suitable for deployment or local demonstration. The README explains both backend and frontend setup, confirming that the repository is organized for reproducible execution.

Evaluation of the project shows that the implemented modules match the original objective set. Dynamic event management is available through the organizer dashboard. Registration immediately produces a digital ticket. Verification produces attendance records in real time. Certificate generation is integrated into the event completion workflow. In other words, the system completes the complete academic event lifecycle rather than stopping after registration.

The code also demonstrates thoughtful design choices. For instance, the event controller supports filtering, keyword search, date ranges, and pagination. The ticket controller distinguishes between preview and download operations. The frontend maintains state for filters, tabs, and previews while preserving responsive card layouts. These elements collectively indicate a project of sufficient technical depth for a BCA final semester submission.

Algorithmic Flow for Ticket Verification

```
1. Accept ticketId and eventId from the verification request.
2. Reject the request if any required field is missing.
3. Fetch the ticket and populate related event and user data.
4. Reject the request if the ticket does not exist.
5. Reject the request if the ticket belongs to another event.
6. Reject the request if the logged-in organizer does not own the event.
7. Reject the request if the ticket status is USED or CANCELLED.
8. Mark the ticket as USED and store usedAt timestamp.
9. Create one attendance record for the attendee and event.
10. Return a success payload with attendee details and entry time.
```

Containing Maintenance Measures

Maintenance measures have been considered from both code and deployment perspectives. The codebase is modular and directory driven, making it easier to update one concern without destabilizing unrelated functionality. Clear model definitions and enums reduce ambiguity when new statuses or event types must be introduced. Utility functions centralize PDF and QR behavior, avoiding duplicated logic across controllers.

Preventive maintenance includes keeping dependencies updated, reviewing environment variable handling, backing up database data, and testing routes after package upgrades. Corrective maintenance can be performed by tracing logs around controllers and services. Perfective maintenance may include enhanced analytics, improved search, notification integration, or administrative controls. Adaptive maintenance may include altering deployment settings or storage strategies for a different college environment.

Conclusion

UniEvent successfully addresses the assigned problem of digitizing university event operations in a dynamic and interactive manner. The final application is not restricted to static pages or theoretical diagrams; it performs live registration, ticket generation, verification, attendance capture, and certificate issue. The project therefore meets the academic requirement that the solution be dynamic, database-driven, and functionally rich.

From a technical perspective, the project demonstrates competence across frontend development, backend API design, data modeling, security practices, document generation, and modular architecture. From an institutional perspective, it reduces manual paperwork, improves entry accuracy, and gives students a more dependable and modern event experience. It is suitable for demonstration, future extension, and real departmental use with minor deployment configuration.

Future Scope of the Project

- Email and WhatsApp notifications for registration confirmation, reminder alerts, and certificate availability.
- Administrative dashboards for cross-department oversight and institute-wide analytics.
- Attendance export to spreadsheet or PDF summary formats for departmental records.
- Waitlist management for full events and auto-promotion when tickets are cancelled.
- Role extension for faculty coordinators, volunteers, and super administrators.
- Deeper reporting such as department-wise participation trends and event conversion statistics.
- Native mobile client or PWA enhancements for more reliable offline scanning.

Annexure

This annexure contains selected supporting material, source-code extracts, module inventories, API references, and explanatory notes that complement the implementation chapters. The annexure is particularly useful for viva-voce preparation because it gives concise references that can be discussed quickly without opening the full repository during presentation.

Module Inventory

Module	Key Files	Purpose
Authentication	auth.routes.js, auth.controller.js, auth.middleware.js	Signup, login, password reset, profile access, authoriz
Event Management	event.routes.js, event.controller.js, Event.model.js	Create, list, update, delete, publish, complete, and que
Ticketing	ticket.routes.js, ticket.controller.js, Ticket.model.js	Registration, ticket retrieval, preview/download, verific
Attendance	attendance.routes.js, attendance.controller.js, Attendance.model.js	Attendance capture and event-wise attendance statist
Certificates	certificate.routes.js, certificate.controller.js, certificate.service.js	Generate, list, preview, and download certificates.
Frontend Dashboards	student/page.tsx, organizer/page.tsx	Role-specific interaction flows and data presentation.

API Reference Snapshot

HTTP Method	Endpoint	Purpose
POST	/api/auth/signup	Create a new student or organizer account.
POST	/api/auth/login	Authenticate a user and return access credentials.
GET	/api/events	List published events or organizer-owned events.
POST	/api/events	Create a new event.
PUT	/api/events/:id	Update event details or status.
POST	/api/tickets/register/:eventId	Register current student for an event.
POST	/api/tickets/verify	Verify QR or manual ticket input at the venue.
GET	/api/tickets/:ticketId/download	Download ticket PDF.
POST	/api/certificates/generate/:eventId	Generate certificates for an event.
GET	/api/certificates/:certificateId/download	Download certificate PDF.

Additional Source Listing

Auth route validation

```
/**
 * Authentication Routes
 * Handles user registration, login, and profile
 */

```

```
 6  const express = require('express');
 7  const router = express.Router();
 8  const { body } = require('express-validator');
 9  const authController = require('../controllers/auth.controller');
10  const { authenticate } = require('../middleware/auth.middleware');
11  const validateRequest = require('../middleware/validateRequest');
12  const { authLimiter } = require('../middleware/rateLimiters');
13  const { stripTags } = require('../utils/sanitize');
14
15  // Validation rules
16  const signupValidation = [
17    body('name')
18      .trim()
19      .customSanitizer(stripTags)
20      .isLength({ min: 2, max: 100 })
21      .withMessage('Name must be between 2 and 100 characters'),
22    body('email')
23      .isEmail()
24      .withMessage('Please provide a valid email')
25      .normalizeEmail(),
26    body('password')
27      .isString()
28      .isLength({ min: 6, max: 128 })
29      .withMessage('Password must be between 6 and 128 characters'),
30    body('department')
31      .trim()
32      .customSanitizer(stripTags)
33      .isLength({ min: 2, max: 100 })
34      .withMessage('Department must be between 2 and 100 characters'),
35    body('role')
36      .optional()
37      .isIn(['student', 'organizer'])
38      .withMessage('Invalid role'),
39    // Roll number is required for students (matches controller behavior)
40    body('rollNumber')
41      .if(body('role').equals('student'))
42      .trim()
43      .customSanitizer(stripTags)
44      .isLength({ min: 1, max: 30 })
45      .withMessage('Roll number is required for students')
46  ];
47
48  const loginValidation = [
49    body('email')
50      .isEmail()
51      .withMessage('Please provide a valid email')
52      .normalizeEmail(),
53    body('password')
54      .isString()
55      .notEmpty()
56      .withMessage('Password is required')
57      .isLength({ max: 128 })
58      .withMessage('Password is too long')
59  ];
60
61  const forgotPasswordValidation = [
62    body('email')
63      .isEmail()
64      .withMessage('Please provide a valid email')
65      .normalizeEmail(),
66  ];
67
68  const resetPasswordValidation = [
69    body('password')
70      .isString()
71      .isLength({ min: 6, max: 128 })
72      .withMessage('Password must be between 6 and 128 characters')
73  ];
74
75  // Routes
76  // Rate limit ONLY auth endpoints (do not rate-limit other APIs)
77  router.post('/signup', authLimiter, signupValidation, validateRequest, authController.signup);
78  router.post('/login', authLimiter, loginValidation, validateRequest, authController.login);
79  router.post('/forgot-password', authLimiter, forgotPasswordValidation, validateRequest, authController.forgot
80  router.post('/reset-password/:token', authLimiter, resetPasswordValidation, validateRequest, authController.r
81  router.get('/me', authenticate, authController.getProfile);
82
83  module.exports = router;
```

Ticket model

```
/**
 * Ticket Model
 * Stores ticket information with QR codes
 *
 * SECURITY NOTE: QR code contains ONLY the ticketId
 * No personal data is stored in the QR code
 */

const mongoose = require('mongoose');
const { v4: uuidv4 } = require('uuid');

// Ticket status
const TICKET_STATUS = {
  UNUSED: 'unused',
  USED: 'used',
  CANCELLED: 'cancelled'
};

const ticketSchema = new mongoose.Schema({
  ticketId: {
    type: String,
    unique: true,
    default: () => `TKT-${uuidv4().slice(0, 8).toUpperCase()}`
  },
  eventId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Event',
    required: [true, 'Event is required']
  },
  userId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: [true, 'User is required']
  },
  qrCode: {
    type: String,  // Base64 encoded QR code image
    required: true
  },
  status: {
    type: String,
    enum: Object.values(TICKET_STATUS),
    default: TICKET_STATUS.UNUSED
  },
  usedAt: {
    type: Date,
    default: null
  }
}, {
  timestamps: true
});

// Compound index for efficient lookups
ticketSchema.index({ eventId: 1, userId: 1 }, { unique: true });
// ticketSchema.index({ ticketId: 1 });
ticketSchema.index({ status: 1 });

/**
 * Check if ticket can be used for entry
 */
ticketSchema.methods.canBeUsed = function() {
  return this.status === TICKET_STATUS.UNUSED;
};

/**
 * Mark ticket as used
 */
ticketSchema.methods.markAsUsed = async function() {
  this.status = TICKET_STATUS.USED;
  this.usedAt = new Date();
  return this.save();
};

// Static constants
```

```
ticketSchema.statics.TICKET_STATUS = TICKET_STATUS;

const Ticket = mongoose.model('Ticket', ticketSchema);

module.exports = Ticket;
```

Attendance model

```
/**
 * Attendance Model
 * Tracks attendance for events
 *
 * Attendance is automatically recorded when a ticket is scanned and verified
 */

const mongoose = require('mongoose');

const attendanceSchema = new mongoose.Schema({
  eventId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Event',
    required: [true, 'Event is required']
  },
  userId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: [true, 'User is required']
  },
  ticketId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Ticket',
    required: [true, 'Ticket is required']
  },
  entryTime: {
    type: Date,
    default: Date.now
  },
  verifiedBy: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',  // Organizer who verified the ticket
    required: true
  }
}, {
  timestamps: true
});

// Compound index - one attendance record per user per event
attendanceSchema.index({ eventId: 1, userId: 1 }, { unique: true });
attendanceSchema.index({ eventId: 1, entryTime: 1 });

const Attendance = mongoose.model('Attendance', attendanceSchema);

module.exports = Attendance;
```

Certificate model

```
/**
 * Certificate Model
 * Stores certificates generated after event completion
 *
 * Certificates are typically generated for workshops
 * and other educational events where attendance matters
 */

const mongoose = require('mongoose');
const { v4: uuidv4 } = require('uuid');

const certificateSchema = new mongoose.Schema({
  certificateId: {
    type: String,
    unique: true,
    default: () => `CERT-${uuidv4().slice(0, 12).toUpperCase()}`
  },
  eventId: {
```

```
19      type: mongoose.Schema.Types.ObjectId,
20      ref: 'Event',
21      required: [true, 'Event is required']
22    },
23    userId: {
24      type: mongoose.Schema.Types.ObjectId,
25      ref: 'User',
26      required: [true, 'User is required']
27    },
28    issuedAt: {
29      type: Date,
30      default: Date.now
31    },
32    issuedBy: {
33      type: mongoose.Schema.Types.ObjectId,
34      ref: 'User',  // Organizer who issued the certificate
35      required: true
36    },
37    pdfUrl: {
38      type: String,
39      default: null
40    },
41    pdfFileId: {
42      type: mongoose.Schema.Types.ObjectId,
43      default: null
44    },
45    filePath: {
46      type: String,
47      default: null  // Deprecated: previously used for local filesystem storage
48    }
49  }, {
50    timestamps: true
51  });
52
53  // Compound index - one certificate per user per event
54  certificateSchema.index({ eventId: 1, userId: 1 }, { unique: true });
55  // certificateSchema.index({ certificateId: 1 });
56  certificateSchema.index({ userId: 1 });
57
58  const Certificate = mongoose.model('Certificate', certificateSchema);
59
60  module.exports = Certificate;
```

Supplementary Technical Note 1

This supplementary note is included to maintain a strong minimum body length for the formal project report while still staying relevant to the implemented project. Note 1 elaborates how the modular repository structure supports ongoing maintenance and evaluation in an academic environment.

The backend source tree separates config, controllers, middleware, models, routes, services, and utilities. This decomposition supports targeted review during viva-voce because a panel can ask about validation, persistence, or business logic independently. The frontend mirrors the same clarity by grouping pages, components, hooks, context providers, and shared utilities.

From an educational perspective, this structure demonstrates that the project is not a monolithic script but a maintainable application. Each folder communicates intent, helps with debugging, and makes later changes safer. Such structure is an important sign of software engineering maturity in a final-semester project.

Another important point is that document generation is embedded into the application rather than outsourced to manual office work. Ticket and certificate PDFs are produced programmatically, ensuring consistency in format and reducing repetitive effort for organizers. This aligns the project closely with actual institutional needs.

Supplementary Technical Note 2

This supplementary note is included to maintain a strong minimum body length for the formal project report while still staying relevant to the implemented project. Note 2 elaborates how the modular repository structure supports ongoing maintenance and evaluation in an academic environment.

The backend source tree separates config, controllers, middleware, models, routes, services, and utilities. This decomposition supports targeted review during viva-voce because a panel can ask about validation, persistence, or business logic independently. The frontend mirrors the same clarity by grouping pages, components, hooks, context providers, and shared utilities.

From an educational perspective, this structure demonstrates that the project is not a monolithic script but a maintainable application. Each folder communicates intent, helps with debugging, and makes later changes safer. Such structure is an important sign of software engineering maturity in a final-semester project.

Another important point is that document generation is embedded into the application rather than outsourced to manual office work. Ticket and certificate PDFs are produced programmatically, ensuring consistency in format and reducing repetitive effort for organizers. This aligns the project closely with actual institutional needs.

Bibliography

- Project repository source code: frontend and backend modules in the local UniEvent workspace.
- README.md included with the project repository for setup details and API overview.
- Next.js official documentation for App Router concepts and deployment practices.
- React official documentation for component architecture and hooks usage.
- Express.js official documentation for routing and middleware organization.
- MongoDB and Mongoose documentation for schema design, indexing, and query handling.
- JSON Web Token documentation for token-based authentication concepts.
- PDFKit documentation for server-side generation of formal ticket and certificate documents.
- QRCode library documentation for QR image generation in Node.js.